Understanding Robustness Limits of Learned Cardinality Estimators: A Worst-Case Data Distribution Analysis

Yingze Li, Xianglong Liu, Dong Wang, Kaixin Zhang, Zhiyu Liang, Xingyue Wang, Wu Ben, Hongzhi Wang*
Harbin Institute of Technology
China
23B903046@stu.hit.edu.cn,wangzh@hit.edu.cn

ABSTRACT

Learned cardinality estimators have achieved superior accuracy, yet their inherent fragility to data drift remains a primary barrier to production deployment. Existing robustness evaluations typically rely on simplified drift assumptions, failing to bound performance degradation under the complex, non-random logic of real-world workloads. This lack of reliability guarantees necessitates a worst-case analysis to determine how minor data perturbations can maximally compromise the integrity of the learned distribution.

While the susceptibility of models to data drift is widely acknowledged, identifying the minimal drift that maximizes performance degradation is fundamentally challenging; we prove that finding such an optimal adversarial drift is NP-Hard. To bridge this gap, we develop a polynomial-time algorithm with a theoretical $(1-\kappa)$ approximation guarantee. Our analysis reveals a severe structural vulnerability: unlike prior benchmarks requiring extensive shifts (e.g., $>20\%$) to trigger decay, DSA demonstrates that perturbing a targeted subset of merely $\approx 0.8\%$ of tuples suffices to inflate 90th-percentile Q-errors by three orders of magnitude. This shift triggers up to a 20× increase in query latency on STATS-CEB and IMDB-JOB. Finally, we leverage DSA to guide five distinct robustness designs—ranging from maintenance sentinels to architectural hardening—to mitigate such worst-case degradation.

1 INTRODUCTION

Cardinality Estimation (CE) plays a crucial role in query optimization within database systems, as it predicts the number of tuples that satisfy a query without execution and aids the database optimizer select the best query execution plan with the lowest cost [23], thus improving system efficiency[30, 56]. Compared to traditional estimators [43, 52], learned cardinality estimators offer more accurate estimations and have therefore garnered extensive attention in recent years [32, 39, 47, 65]. These learned approaches are generally categorized into Data-driven approaches [32, 65], Query-driven approaches [26, 39] and Hybrid approaches [47, 50, 58] all of which outperform traditional methods by leveraging learned patterns from data or workloads.

Despite their empirical success, the integration of learned estimators into production database kernels remains bottlenecked by concerns regarding their stability and robustness in dynamic environments [30, 47, 50, 69]. While existing literature suggests that learned estimators can tolerate modest distribution shifts under specific, restricted patterns (e.g., random/sequential updates) [47, 50, 56], their behavior under complex real-world perturbation logic remains largely unexplored [57]. Consequently, the "worst-case" performance of these models under data evolution remains a black box [45, 69]. To build a truly resilient database kernel, rigorous mechanisms are required to audit and bound estimator performance against adversarial or complex data drifts [22, 68].

To date, robustness research has primarily focused on workload-centric sensitivity analysis. PACE [69], proposed by Zhang et al., involves adding noise to query workloads and stress-testing the query-driven estimators. This degrades the accuracy of query-driven models and slows down the end-to-end performance. Although workload-centric analysis can significantly impair the performance of query-driven estimators, we find that they have limited impact on widely used data-driven learned estimators [32, 38, 48, 55, 59, 60, 64, 65, 75]. This is primarily because data-driven methods utilize data-level information that is independent of historical workloads to achieve robust cardinality estimation across diverse testing query distributions. Consequently, these methods can effectively withstand perturbations that target historical workloads. Therefore, workload-centric methodology is not suitable for evaluating the robustness limits of these prevalent data-driven models.

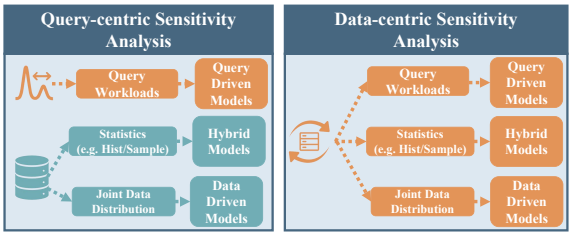


Figure 1: Workload-centric Sensitivity Analysis (Left) vs. Data-centric Sensitivity Analysis (Right).

Notably, data updates inevitably degrade all learned cardinality estimators [41, 42, 46, 47, 55], as these models rely on static training statistics (e.g., historical workloads [26, 39] or data distributions [32, 59, 64]) that become erroneous as underlying properties shift [37, 42, 47]. To evaluate robustness of data drifts, prior works typically rely on simplified drift simulations: Robust-MSCN [50] mimics temporal updates via sequential partitioning, while ALECE [47] employs random shuffling or broad insertions (e.g., $>20\%$ of the entire database). However, real-world update logic is often complex and skewed [20, 27, 36, 53] rather than random or sequential.

This discrepancy obscures a critical blind spot: *can small-scale updates precipitate significant distribution shifts and cause performance collapses of optimizer?* Leaving this question unanswered precludes

the stability guarantees essential for production adoption, users will simply reject a system prone to unpredictable latency spikes under small scale updates. Motivated by this reliability concern, we conduct a worst-case sensitivity analysis to audit how minimal but specific data drifts maximally degrade all outdated estimators.

However, constructing such a universal auditor to identify the smallest worst-case data drift is non-trivial. It requires overcoming three fundamental challenges: (1) **The Necessity of Black-box Universality.** To uncover universal weaknesses across diverse estimators, we require an architecturally agnostic framework. Relying on white-box analyses tailored to specific mechanisms [40, 63, 69, 71] inherently limits generalization and fails in production environments where model details are opaque [8, 35]. Thus, a strict prior-free black-box assumption is essential for holistic reliability guarantees. (2) **Tractability Awareness.** Can the maximum damage of this drift be pre-calculated in polynomial-time? If computationally feasible, such analysis allows us to proactively evaluate the worst-case behavior of all learned estimators, acting as a theoretical canary in a coal mine' to alert us to potential worst-case scenarios in advance. (3) **Limited Budget.** How can a limited number of data modifications on the training data maximally degrade the accuracy of models trained on this perturbed dataset? Since small-scale data modifications seem to have limited impact on learned estimators. For instance, prior studies have shown that outdated models can tolerate random data-level changes exceeding 30% while still maintaining strong estimation performance [32, 41, 56].

According to the above discussions, the toughest challenge lies in that the auditor has no information about the estimator deployed internally within the black-box, which hinders the auditor from effectively quantifying and boosting the efficiency of the stress test. To address this issue, we discovered a crucial and often overlooked fact: learned estimators are approaching oracle-level accuracy on training datasets, obtaining accurate cardinality. For example, many learned estimators [32, 47, 55, 60, 65, 75] achieve more than 90% of the Qerrors close to 1. However, this primary advantage also exposes these models to a common weakness. Despite their varying model types, the predictive output behavior of each model tends to closely mirror the oracle's behavior within the training dataset.

Therefore, we can leverage this exposed common weakness to address the aforementioned challenge. Instead of directly tackling the black-box analysis head-on without necessary information, we can indirectly construct a finite set of data drifts designed to shift the statistics of the oracle in the training database, and thereby impact the performance of all learned estimators. Thus, to overcome the first black-box challenge, we utilize the oracle based on the training data as our surrogate model and convert the difficult black-box audit problem into a tuple selection problem with much more information. To tackle the second tractability challenge, we formulate this data-centric sensitivity analysis as a combinatorial optimization problem and establish that this problem is NP-Hard. To achieve the most effective stress test within a limited budget and address the third challenge, we analyze the supermodular properties of the optimization objective under specific conditions and design a $(1-\kappa)$ approximation algorithm for efficient resolution.

Contributions: We make the following contributions.

C1: **Black-box Sensitivity Analysis on All Estimators**: We investigate how to conduct a successful Data-centric Sensitivity Analysis (DSA) that uses minimum data drifts to maximally degrade the performance of all outdated estimators in a black-box setting (§ 3.2).

C2: **Intractability Insights**: We demonstrate that, under a limited budget, finding the optimal data drift that maximally impacts all estimators is NP-Hard, even when only considering deletion perturbation operations (§ 3.3).

C3: **Near-Optimal Perturbation Strategy**: We design a polynomial-time approximation algorithm (§ 4.2) that guarantees a $(1-\kappa)$ approximation for the worst-case performance of all estimators. This analysis acts as a theoretical early warning mechanism, allowing us to proactively quantify and approximate the worst-case scenarios of all learned estimators (§ 4.3).

C4: **Quantifying Limits & Robustness Designs**: We expose a critical sensitivity gap: modifying merely **0.8%** of the database triggers a 1000× error spike and raises latency by up to 20×. Beyond auditing, we utilize DSA to inspire five distinct robustness designs, transforming theoretical bounds into practical defenses against worst-case drifts.

2 PRELIMINARIES

We present learned cardinality estimators in § 2.1, algorithmic complexity sensitivity analysis for learned DBMS components in § 2.2, and our formal stress testing framework in § 2.3.

2.1 Learned Cardinality Estimation

Suppose a database D consists ℓ relations, i.e. $D = \{R_1, R_2, \dots R_\ell\}$. Given a query $\mathbf{q}$, when $\mathbf{q}$ is executed on D, we obtain $\mathbf{q}$'s results in result set $\mathbb{S}(\mathbf{q}|D)$. We denote the cardinality of $\mathbb{S}(\mathbf{q}|D)$ as $C_D(\mathbf{q})$.

Tuple's Joint Weight: For relation R_x with N tuples, a tuple $t_i \in R_x$ and a query $\mathbf{q}_j$, the joint weight w_{ij} denotes tuple t_i's contribution to query $\mathbf{q}_j$'s result's cardinality, that is $w_{ij} = |\mathbb{S}(\mathbf{q}_j|D - R_x + \{t_i\})|$. $\mathbf{q}_j$'s cardinality can be seen as a linear summation of R_x's tuple's the joint weights: $C_D(\mathbf{q}_j) = \sum_{i=1}^{N} w_{ij}$.

Learned Cardinality Estimation: Learned cardinality estimation needs to make predictions on $C_D(\mathbf{q})$ using learned estimator E without executing $\mathbf{q}$ on D. E is trained via the information of D. Based on the training information, these learned cardinality estimators can be categorized into these main paradigms: data-driven, query-driven, and hybrid.

Learned Data-driven Estimator: Learned data-driven cardinality estimator E_{Data} learns the joint distribution of database D. Based on the learned statistical information, E_{Data} provides the query independent estimation of the query $\mathbf{q}$'s cardinality $est(\mathbf{q}|E_{Data})$.

Learned Query-driven Estimator: Learned query-driven cardinality estimator E_{Query} learns the mapping from historical workloads Q to cardinalities C, i.e., $Q \rightarrow C$. Based on the learned mapping knowledge, E_{Query} provides the estimation of the query $\mathbf{q}$'s cardinality $est(\mathbf{q}|E_{Query})$.

Hybrid estimator: Learned hybrid estimator E_{Hybrid} combines query Q with statistics (e.g. histograms [47], samples [39]) $\mathcal{S}$ as the input features, and learns the mapping $\{Q \times \mathcal{S}\} \rightarrow C$. Based on the enhanced features, E_{Hybrid} provides a comprehensive estimation of query $\mathbf{q}$'s cardinality $est(\mathbf{q}|E_{Hybrid})$ compared to $est(\mathbf{q}|E_{Query})$.

For query $\mathbf{q}_j$, we follow the assumptions made in the majority of CE literature [30, 38, 39, 47, 64], considering $\mathbf{q}_j$ to be the most prevalent selection-projection-join query of the form:

```
SELECT * FROM q_j.Tabs WHERE q_j.Joins AND q_j.Filters
```

where: $\mathbf{q}_j.Tabs$ denotes the set of tables involved in the query $\mathbf{q}_j$, such that $\mathbf{q}_j.Tabs \subseteq D$, and $\mathbf{q}_j.Joins$ represents the join predicates, $\mathbf{q}_j.Filters$ signifies the filter predicates.

2.2 From Simple Drift to Adversarial Drift: The Necessity of Adversarial Stress Testing

To ensure learned database components work reliably in production, thorough robustness evaluation is essential [56]. However, existing evaluation methodologies largely rely on simulating data updates via random drift or simple sequential patterns [47, 56]. While these methods confirm that learned estimators perform well under "average" jitter, they fundamentally fail to stress-test the system against *worst-case* distribution shifts. This limitation is critical because real-world data drifts are rarely purely random; they often stem from structured business logic (e.g., regional inventory purges or targeted user churn) [57], which can create correlated biases that hit the specific weak points of a model. Relying solely on random testing creates a false sense of security, leaving the system's theoretical lowest performance bound explorative and quantifying the "tail risks" unaddressed.

To bridge this gap and identifying the true stability boundaries of learned estimators, we draw inspiration from the methodology of Algorithmic Complexity Attacks (ACA) [10, 19, 51]. Originally framed as a security vector to exhaust system resources, the mathematical core of ACA is an *optimization problem*: finding the specific input permutation that maximizes algorithmic complexity. We adapt this adversarial philosophy not to compromise the system, but to conduct a rigorous Robustness Audit. Just as Kornaropoulos et al. [63] utilized this mindset to uncover memory failure modes in learned indexes, we apply adversarial optimization to mathematically locate the "Achilles' heel" of cardinality estimators—the specific data subset whose modification causes maximum degradation.

We formalize this approach as a Data-centric Sensitivity Analysis (DSA). Unlike PACE [69], which focuses on query workload shifts, DSA targets the fundamental data distribution, auditing both data-driven and query-driven models under data drifts. Fundamentally, the database optimizer ***decreases*** the computational complexity of physical plans by utilizing cost estimates [23]. Conversely, DSA seeks to maximize this complexity by identifying the worst-case data perturbations that mislead the estimator most severely. By solving this adversarial optimization problem, DSA allows us to calculate the theoretical performance floor of the system. The practical necessity of this worst-case analysis is demonstrated in the following scenario:

Example 2.1. *Figure 2 illustrates the core mechanism of DSA. Consider estimators initially trained on a stale database snapshot (Step 1). To probe the fundamental sensitivity limit, instead of random updates, we identifies* ***a single, targeted tuple update*** *(setting $R_x.ID = 2$) in Step 2. Crucially, while a random update might be statistically negligible, this targeted update hits the model's specific knowledge boundary. As a result, in Step 3, the estimators remain anchored to outdated statistics (predicting a cardinality near 0), failing to adapt to the critical data change. This discrepancy misleads the query optimizer in Step 4, causing it to erroneously favor a Nested Loop join over a more efficient Hash Join. The consequence, shown for JOB-Q60 in Figure 2 (right), is severe: the algorithmic complexity degrades from the Hash Join's optimal $O(M+N)$ to the Nested Loop's prohibitive $O(M \times N)$, exploding the execution time by over 9.2×10^4 seconds. This case demonstrates that system fragility is not defined by how much data changes, but by which data changes. DSA allows us to identify these critical weak points.*

2.3 Stress Testing Framework

In this section, we establish the stress testing framework via worst-case sensitivity analysis. We formalize the model by defining the auditor, their goals, knowledge, and sensitivity evaluation metrics.

Auditor and Auditor's Goal: To investigate the worst-case performance of all cardinality estimators in dynamically changing databases, we postulate a virtual auditor performing a stress test that emerges when the estimator's information becomes outdated. To uniformly analyze the impact of different perturbation strategies under consistent distributions, we fix the updated database state as D. The auditor aims to find minimized critical modifications ΔD (insertions/deletions) such that models trained on the perturbed distribution $D' = D + \Delta D$ yield catastrophically inaccurate estimations when applied to the original version D. This misleads the optimizer to select physical plans with the highest cost when evaluated on the original version D. For brevity, we refer to D as the *original state* and D' as the *perturbed state*. We follow the established convention of query optimization research [21, 28, 29] by interpreting each base relation under standard SQL bag semantics. In this context, tuples may appear with specific multiplicities, and a perturbation ΔD is permitted to consist of the insertion or deletion of logical duplicates.

Auditor's Knowledge: In this work, we study a worst-case sensitivity analysis setting where the auditor (e.g., a system administrator or a robustness testing tool) has read-only access to the original database snapshot D and the schema. The auditor has no information about the internal architecture or parameters of the deployed learned estimator, except that it is trained on a perturbed snapshot D'. The auditor is given a test workload W_{Test} (defined below) and can execute diagnostic aggregate queries on D to compute the statistics needed for sensitivity analysis.

Auditor's Capacity: We assume the auditor can issue diagnostic SQL queries on D. For each query $q_j \in W_{Test}$ and a target relation R_x with primary key PK_x, the auditor can compute the joint weight w_{ij} of each tuple $t_i \in R_x$ by rewriting q_j into a grouped counting query of the form:

```
SELECT R_x.PK_x, COUNT(*) FROM ... GROUP BY R_x.PK_x,
```

which returns, for each t_i, the number of output tuples contributed by t_i to q_j on D. This capability does not require tuple lineage from the final query result; it only assumes standard read-only privileges to run aggregate diagnostics, which is realistic for an auditor. Additionally, the auditor can apply at most K update operations (insertions/deletions) to a specific relation R_x to form a perturbed snapshot $D' = D + \Delta D$ for evaluating robustness under a limited budget. When the production database enforces primary-key or foreign-key constraints, we assume the auditor operates on an offline copy or shadow tables where such multiplicities are encoded via surrogate identifiers or relaxed integrity constraints; this ensures that query answers and optimizer behavior are preserved while our stress test manipulates tuple multiplicities. We explicitly restrict modifications to a single relation to isolate the causal impact of data drift from the confounding factors of complex multi-table join interactions. While multi-table perturbations represent a

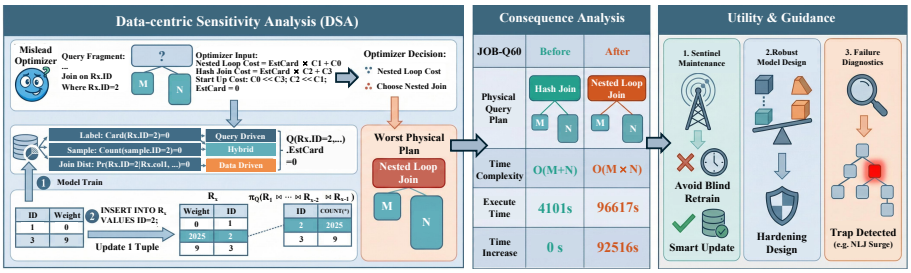


Figure 2: DSA workflow (Left); The consequence of DSA (Right).

broader challenge, demonstrating that catastrophic regression can be induced even under such limited conditions is sufficient to expose the fundamental fragility of the estimators. This design choice provides a rigorous, interpretable lens for precise root-cause analysis.

Sensitivity Evaluation Metric. The auditor has to devise specific strategies to impair these commonly used metrics: (1) **Qerror metric**: Defined as $Q(est(\mathbf{q}|E), C_D(\mathbf{q})) = max(\frac{est(\mathbf{q}|E)}{C_D(\mathbf{q})}, \frac{C_D(\mathbf{q})}{est(\mathbf{q}|E)})$, where $est(\mathbf{q}|E)$ is the estimators E's prediction on given query $\mathbf{q}$. It measures the distance between the estimated cardinality $est(\mathbf{q}|E)$ and the true cardinality $C_D(\mathbf{q})$ of a query. (2) **End-to-end latency**: Measures the total latency for generating a plan with estimated cardinalities and executing the physical plan. This metric is essential for demonstrating how an estimator can improve query optimization performance. It serves as a gold standard for assessing the effectiveness of CE approaches [30, 38, 47, 59].

3 BLACK-BOX SENSITIVITY ANALYSIS FOR ALL LEARNED ESTIMATORS

In this section, we will apply the concepts of data-centric sensitivity analysis and the stress testing framework established in § 2 to define the black-box data-centric sensitivity analysis in § 3.1. We will appropriately transform this problem into a constrained integer nonlinear programming problem in § 3.2 and analyze its computational intractability in § 3.3.

3.1 Black-box Data-centric Sensitivity Analysis

In this section, we provide the definition of black-box Data-centric Sensitivity Analysis (DSA) and introduce the challenge associated with directly solving this problem. We assume that the auditor can execute at most K tuple-level data modifications ΔD on the original database state D, simulating a data drift scenario where the training state shifts to $D' = D + \Delta D$. The auditor aims to ensure that, when evaluated on the testing workload $W_{Test} = \{\mathbf{q}_j | 1 \le j \le M\}$ with M queries, the estimator E, trained on the perturbed database D', has the worst accuracy on the original database D, i.e., maximize Eq. 1:

$$\text{Max}: \sum_{j=1}^{M} Q(est(\mathbf{q}_j|E), C_D(\mathbf{q}_j)) = \sum_{j=1}^{M} \max\left(\frac{est(\mathbf{q}_j|E)}{C_D(\mathbf{q}_j)}, \frac{C_D(\mathbf{q}_j)}{est(\mathbf{q}_j|E)}\right) \tag{1}$$

$$\text{s.t. } (1)D' = D + \Delta D \quad (2)E \text{ is trained on } D' \quad (3)|\Delta D| \le K$$

When maximizing Eq. 1, we assume a universal scenario, where the auditor don't need to acquire knowledge of the internal estimator type used in the database. Specifically, while the auditor knows that E is trained on D', they are unaware of whether E is data-driven, query-driven, or hybrid. Additionally, the black-box setting restricts the auditor's visibility by concealing details of the deployed model, introducing significant challenges. The internal models could include MLPs [26, 65], CNNs [39, 50], Transformers [47, 64], or Bayesian networks [59, 60]. Consequently, the auditor cannot leverage existing white-box evaluation methodologies from ML security [12, 15, 16, 34, 61, 62, 70]. This renders direct black-box data-centric DSA seemingly infeasible, necessitating a novel approach to reframe the problem.

3.2 Targeting the Surrogate Oracle

In this section, we will take a different perspective to transform the seemingly impossible black-box DSA problem from the previous section into a constrained integer nonlinear programming problem that can be tackled. Specifically, we are inspired by the experimental results of learned CE studies [30, 47, 56, 64, 65], which show that existing learned estimators possess extremely high estimation accuracy, with their estimates on the training database approaching oracle-level. Therefore, we utilize an oracle estimator E_O on the dataset D' as our surrogate model. We assume that the E_O trained on D' can precisely report the actual cardinality of queries in D'. For E_O, we have $est(\mathbf{q}|E_O) = C_{D'}(\mathbf{q})$. Thus, our task now is to maximize the Qerror of the oracle trained on D':

$$\text{Max}: \sum_{j=1}^{M} \max\left(\frac{est(\mathbf{q}_j|E_O)}{C_D(\mathbf{q}_j)}, \frac{C_D(\mathbf{q}_j)}{est(\mathbf{q}_j|E_O)}\right) = \sum_{j=1}^{M} \max\left(\frac{C_{D'}(\mathbf{q}_j)}{C_D(\mathbf{q}_j)}, \frac{C_D(\mathbf{q}_j)}{C_{D'}(\mathbf{q}_j)}\right) \tag{2}$$

$$\text{s.t. } (1)D' = D + \Delta D \quad (2)|\Delta D| \le K$$

Given the test workload W_{Test} and a relation R_x of N tuples, denoted as $R_x = \{t_1, t_2, \ldots, t_N\}$. We consider using $t_i.\beta$ to represent the auditor's behavior on tuple t_i, where $t_i.\beta$ takes -1 to indicate delete tuple t_i from the R_x, $t_i.\beta$ takes 0 to indicate no operation on tuple t_i, and $t_i.\beta$ takes k to indicate duplicating tuple t_i by k times and re-inserting them into the database. Therefore, for the poisoned database D', the cardinality of query $\mathbf{q}_j$ on D' equals to $C_{D'}(\mathbf{q}_j) = C_D(\mathbf{q}_j) + \sum_{i=1}^{N} t_i.\beta \times w_{ij}$. Based on the above, we reorganize the sensitivity analysis problem in Eq. 1 into:

Optimal Data-centric Sensitivity Analysis (Optimal DSA) Problem: Given testing workloads W_{Test}, the auditor needs to provide an modification strategy and maximize the following Eq. 3:

$$\text{Max}: \sum_{j=1}^{M} \max\left(\frac{C_D(\mathbf{q}_j)+1+\sum_{i=1}^{N} t_i.\beta \times w_{ij}}{C_D(\mathbf{q}_j)+1}, \frac{C_D(\mathbf{q}_j)+1}{C_D(\mathbf{q}_j)+1+\sum_{i=1}^{N} t_i.\beta \times w_{ij}}\right) \tag{3}$$

$$s.t. \sum_{i=1}^{N} |t_i.\beta| \le K \quad t_i.\beta \in \{-1, 0, 1, \ldots K\}$$

For the sake of brevity, we employ the abbreviation 'optimal DSA' to represent 'optimal Data-centric Sensitivity Analysis' throughout the remainder of this paper. Meanwhile, we add 1 to both the numerator and denominator when calculating Qerror to avoid division by zero errors. This transformation is a commonly seen evaluation technique in many cardinality estimation methods and can be found in many open-sourced cardinality estimators' GitHub repositories e.g. line 4 in [5], line 21-22 in [6].

In summary, we converted the seemingly unsolvable black-box sensitivity analysis problem stated in Eq. 1 into a constrained integer nonlinear programming problem, as detailed in Eq. 3. However, it remains uncertain whether the auditor can effectively solve Eq. 3 in polynomial-time. Therefore, we will provide an analysis of intractability in the next section.

3.3 Intractability

In this section, we give an intractability analysis of the optimal DSA problem defined in Eq. 3. We conclude that finding the optimal DSA solution is NP-Hard even when considering deletions alone. Additionally, when considering both insertions and deletions, finding an optimal strategy still remains NP-Hard. When only considering data deletions, the problem is converted into the following optimization problem in Eq. 4:

$$\text{Max}: \sum_{j=1}^{M} \frac{C_D(\mathbf{q}_j)+1}{C_D(\mathbf{q}_j)+1+\sum_{i=1}^{N} t_i.\beta \times w_{ij}} \tag{4}$$
$$s.t. \sum_{i=1}^{N} |t_i.\beta| \le K \quad t_i.\beta \in \{-1, 0\}$$

We now present a proof sketch that finds the optimal solution to Eq. 4, is NP-Hard. We start our polynomial-time reduction from the known Densest-K-Subgraph problem, defined as:

Densest-K-Subgraph (DKS) problem: Given a simple, undirected graph $\mathbb{G} = (\mathbb{V}, \mathbb{E})$ and an integer K, find a subset of vertices $S \subseteq \mathbb{V}$ such that: $|S| \le K$ and the subgraph $\mathbb{G}[S]$ induced by S has the maximum number of edges.

The left subfigure in Figure 3 shows a DKS example when $K = 4$. The DKS problem is a well-studied NP-Hard problem in graph theory [9, 17, 18] and closely related to many applications in graph database community such as community search [72–74]. We will next construct a polynomial-time reduction based on this problem.

Theorem 3.1. *The optimal DSA problem defined in Eq. 4 when considering only deletions, is NP-Hard.*

Proof. (Sketch)We establish the NP-Hardness proof by providing a polynomial-time reduction from the DKS problem to the constrained maximization in Eq 4 that finds K deleting tuples to maximize the Qerror.

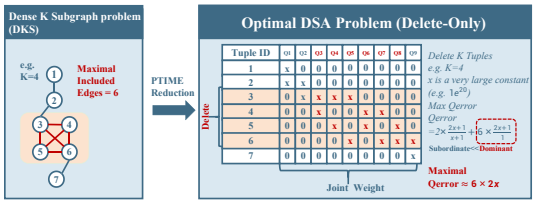


Figure 3: Polynomial-time reduction example starting from Densest-K-Subgraph(DKS) problem.

For any DKS problem P_1, consisting of a graph $\mathbb{G} = (\mathbb{V}, \mathbb{E})$ and a number K, we can always construct a corresponding database instance and a group of testing queries to form the delete-only optimal DSA problem P_2 in Eq 4 in time of $O(|\mathbb{V}| \cdot |\mathbb{E}|)$. The construction is as follows: (1) Each vertex $v_i \in \mathbb{V}$ corresponds to a unique tuple t_i in the relation R_x. (2) Each edge $e_j = \{v_{i1}, v_{i2}\} \in \mathbb{E}$ corresponds to a query $\mathbf{q}_j$ involving only two tuples in R_x, t_{i1} and t_{i2}. (3) We assign each tuple contributes a common join weight $\mathbf{x}$, where $\mathbf{x}$ is a sufficiently large positive integer.

For the constructed delete-only DSA problem P_2, and a given query $\mathbf{q}_j$ consisting of two tuples. If one of its tuples is deleted, the contribution to the objective function is $\Delta Qerr_1 = \frac{2\mathbf{x}+1}{\mathbf{x}+1}$, if two tuples are both deleted, the contribution is $\Delta Qerr_2 = 2\mathbf{x}+1$. When $\mathbf{x}$ is sufficiently large, $\Delta Qerr_1$ approaches 2 and $\Delta Qerr_2$ approaches $2\mathbf{x}$ which is significantly larger: ($\Delta Qerr_2 \gg \Delta Qerr_1$). Consequently, the maximization objective is predominantly influenced by $\Delta Qerr_2$. This implies that maximizing the objective function is effectively equivalent to maximizing $2\mathbf{x}$ times the number of edges in the K-densest subgraph of $\mathbb{G}$. To better illustrate the above ideas, we use Example 3.1 to show the reduction process.

Example 3.1. *Figure 3 presents a reduction example. The left side depicts a DKS problem instance with $K = 4$, while the right side shows a corresponding optimal delete-only DSA problem for $K = 4$. For a graph with 7 vertices and 9 edges, we construct a database instance with a relation R_x containing 7 tuples and 9 queries. For instance, queries $\mathbf{q}_3$, $\mathbf{q}_4$, and $\mathbf{q}_5$ encode connections between vertex 3 and vertices {4, 5, 6}. Each tuple contributes $\mathbf{x}$ to its associated query's cardinality. When $\mathbf{x}$ is sufficiently large (e.g., 10^{20}), deleting one tuple for a single query yields a Qerror contribution of approximately 2, while deleting two tuples results in $2\mathbf{x}$. Thus, the maximum Qerror arises when tuples from two queries are deleted. Moreover, the worst-case Qerror contribution scales with the number of internal edges in the K-vertex subgraph selected by DKS. In our example, the optimal DKS solution is a 4-vertex clique with 6 edges, leading to a maximum Qerror of $6 \times 2\mathbf{x}$ for the delete-only DSA.*

Since the DKS problem is NP-Hard, and we have reduced it to the optimal delete-only DSA problem in polynomial-time, it follows that finding the optimal DSA problem when only considering deletions is NP-Hard. □

Based on the Theorem 3.1, we will demonstrate that considering both insertion and deletion simultaneously, the complete DSA problem is also NP-Hard.

Theorem 3.2. *The optimal DSA problem defined in Eq. 3 when considering both insertions and deletions, is NP-Hard.*

Proof. (Sketch): Our overall proof sketch is as follows: We extend the polynomial-time reduction constructed in Theorem 3.1. When the joint weight $\mathbf{x}$ is larger than $K \times M$, where K is the perturbation budget, and M is the number of queries, the benefits from

I insertion operations($2 \le I \le K$) become negligible compared to delete two tuples within a given query $\mathbf{q}_j$. This leads the auditor to abandon insertion operations and transform the case into a delete-only environment, and we can use the remainder of Theorem 3.1's proof. Therefore, we establish a polynomial-time reduction from the DKS problem to the optimal DSA problem that considers both insertions and deletions, thereby proving that the optimal perturbation strategy involving both insertions and deletions is NP-Hard. We provide rigorous proof in our appendix [7]. □

In summary, computing the exact optimal solution to the DSA problem is NP-hard, meaning that a scalable auditor cannot, in general, certify the worst-case drift by exhaustive search in polynomial time. This result characterizes an inherent computational barrier for worst-case sensitivity analysis and motivates the need for principled approximations. This result motivates us to focus on efficient approximation algorithms: as shown next, auditors can design polynomial-time algorithms with provable guarantees to obtain near-optimal worst-case scenarios.

4 NEAR-OPTIMAL SENSITIVITY ANALYSIS

While the previous section showed that finding universally worst-case perturbations is NP-hard, the DSA problem possesses structural properties that enable efficient solutions. Specifically, § 4.1 analyzes these properties, which we then leverage in § 4.2 to design an approximate algorithm. Finally, § 4.3 provides a rigorous performance analysis of the proposed method.

4.1 Analysis of the DSA's Objective Function

In this section, we will analyze the nature of DSA's objective function under specific conditions to derive certain insights. To facilitate the presentation of symbols in the following discussion, we denote the Qerror induced by the j-th query, considering only deletions, as: $\mathbb{Q}_j(X) = \frac{C_D(\mathbf{q}_j)+1}{C_D(\mathbf{q}_j)+1+\sum_{t_i \in X} t_i.\beta \times w_{ij}}$. X is the deleted tuples within the relation R_x, having $X \subseteq R_x, \forall t_i \in X, t_i.\beta = -1$. Therefore, the total Qerror when only deletion is considered is $\mathbb{Q}(X) = \sum_{j=1}^{M} \mathbb{Q}_j(X)$. Based on these definitions, we have Theorem 4.1

Theorem 4.1. *When only deletion operations are considered, total Qerror $\mathbb{Q}(X)$ of the optimal DSA problem satisfies the supermodular property, that is: $\forall A, B \subseteq R_x, \mathbb{Q}(A \cup B) + \mathbb{Q}(A \cap B) \ge \mathbb{Q}(A) + \mathbb{Q}(B)$.*

Proof. (Sketch)We aim to demonstrate that the objective

$$\mathbb{Q}(X) = \sum_{j=1}^{M} \mathbb{Q}_j(X) = \sum_{j=1}^{M} \frac{C_D(\mathbf{q}_j) + 1}{C_D(\mathbf{q}_j) + 1 + \sum_{t_i \in X} t_i.\beta \times w_{ij}}$$

satisfies the supermodular property. Specifically, for any two sets A and B within R_x, we need to prove that $\forall A, B \subseteq R_x, \mathbb{Q}(A \cup B) + \mathbb{Q}(A \cap B) \ge \mathbb{Q}(A) + \mathbb{Q}(B)$.

First, consider each component function $\mathbb{Q}_j(X)$. We aim to show that $\mathbb{Q}_j(X)$ is supermodular. We denote:

$$w'_{ij} = \frac{w_{ij}}{1 + C_D(\mathbf{q}_j)}, \quad S_j(X) = \sum_{t_i \in X} t_i.\beta \times w'_{ij}$$

Then, the expression:

$$\mathbb{Q}_j(A \cup B) + \mathbb{Q}_j(A \cap B) - \mathbb{Q}_j(A) - \mathbb{Q}_j(B)$$

can be reorganized into the following form, we provide the detailed derivation in the appendix [7].

$$\frac{(2 + S_j(A) + S_j(B))(S_j(A \setminus B))(S_j(B \setminus A))}{(1 + S_j(A \cup B))(1 + S_j(A \cap B))(1 + S_j(A))(1 + S_j(B))}$$

Observe that given $\forall t_i \in X, t_i.\beta = -1, w'_{ij} \ge 0$, we have $S_j(X)$ is negative. We deduced that the two factors of the numerator, $S_j(A \setminus B)$ and $S_j(B \setminus A)$, are both less than or equal to 0. Thus, their product is greater than or equal to 0. Meanwhile, $\forall X \subseteq R_x, (1 + S_j(X)) \ge (1 + S_j(R_x)) = \frac{1}{1+C_D(\mathbf{q}_j)} > 0$. Therefore, each term in the denominator is strictly positive, and $(2 + S_j(A) + S_j(B))$ is strictly positive. In conclusion, within the above fractional, the numerator is greater than or equal to 0, while the denominator is strictly greater than 0. We have:

$$\mathbb{Q}_j(A \cup B) + \mathbb{Q}_j(A \cap B) - \mathbb{Q}_j(A) - \mathbb{Q}_j(B) \ge 0,$$

which confirms that $\mathbb{Q}_j(X)$ is supermodular. Since $\mathbb{Q}(X)$ is the sum of supermodular functions $\mathbb{Q}_j(X)$, it inherits the supermodular property. Formally,

$$\begin{aligned} &\mathbb{Q}(A \cup B) + \mathbb{Q}(A \cap B) - \mathbb{Q}(A) - \mathbb{Q}(B) \\ &= \sum_{j=1}^{M} \left[\mathbb{Q}_j(A \cup B) + \mathbb{Q}_j(A \cap B) - \mathbb{Q}_j(A) - \mathbb{Q}_j(B)\right] \ge 0. \end{aligned}$$

Hence, the objective $\mathbb{Q}(X)$ satisfies the supermodular property. □

Next, we consider the scenario where only insertions are considered, the DSA problem discussed in Eq. 3 can be much simpler. Similar to Theorem 4.1, we define the Qerror induced by the j-th query, considering only insertions, as: $\mathbb{Q}'_j(X) = \frac{C_D(\mathbf{q}_j)+1+\sum_{t_i \in X} t_i.\beta \times w_{ij}}{C_D(\mathbf{q}_j)+1}$. And X is the set of tuples that are repeatedly inserted into R_x, having $X \subseteq R_x, \forall t_i \in X, t_i.\beta = 1$. Therefore, the total Qerror when only insertion is considered is $\mathbb{Q}'(X) = \sum_{j=1}^{M} \mathbb{Q}'_j(X)$. Based on these definitions, we have Theorem 4.2:

Theorem 4.2. *When only considering insertions, the optimal DSA problem satisfies the modular property, i.e., $\forall A, B \subseteq R_x, \mathbb{Q}'(A \cup B) + \mathbb{Q}'(A \cap B) = \mathbb{Q}'(A) + \mathbb{Q}'(B)$.*

Proof. We leave the detailed proof in our appendix [7]. □

The above theorems indicate that although the optimal DSA problem in Eq. 3 is NP-Hard, the objective function possesses special properties when the modification types are limited. We will utilize this characteristic to design an effective approximate perturbation algorithm and analyze the stress test's effectiveness.

4.2 Perturbation Strategy Generation

In this section, we will utilize the characteristics analyzed in the previous section to design a corresponding algorithm to maximize Eq. 3. We first present the perturbation strategy in Algorithm 1:

The general idea of the Algorithm 1 is as follows. In line 1, the auditor performs a group-by operation on the result $\mathcal{R}es$ of the testing queries W_{Test} over the relation R_x and calculates the joint weight of each tuple in R_x with respect to each query join. For a given query $\mathbf{q}_j$ and its materialized result in $\mathbb{S}(\mathbf{q}_j|D)$, this operation can be executed via the following SQL:

```
SELECT R_x.PK, COUNT(*) FROM 𝕊(q_j|D) GROUP BY R_x.PK;
```

where $R_x.PK$ is the primary key of the relation R_x.

Algorithm 1 Approximate solution to optimal DSA.

Require: Table R_x; Testing queries W_{Test}; Results $\mathcal{R}es$; Budget K;
Ensure: Modification set ΔD with at most K modifications;

1: $w = getJointWeight(\mathcal{R}es, R_x)$ ▷ Obtaining joint weight
2: $\Delta D = \phi; Mask = \mathbf{0}_N$
3: **while** $|\Delta D| \leq K$ **do**
4: $bestOp = \phi; bestVal = 0;$
5: **for** $(t_i \in R_x) \wedge (Mask[i] = 0)$ **do** ▷ Get R_x's remained tuples
6: $gainD = \sum_{\mathbf{q}_j \in W_{Test}} Q(C_{D+\Delta D}(\mathbf{q}_j), C_{D+\Delta D}(\mathbf{q}_j) - w_{ij});$
7: **if** $gainD \geq bestVal$ **then**
8: $bestVal = gainD$
9: $bestOp = -t_i$
10: $gainI = \sum_{\mathbf{q}_j \in W_{Test}} Q(C_{D+\Delta D}(\mathbf{q}_j), C_{D+\Delta D}(\mathbf{q}_j) + w_{ij});$
11: **if** $gainI \geq bestVal$ **then**
12: $bestVal = gainI$
13: $bestOp = t_i$
14: **if** $bestOp = \phi$ **then**
15: **break**
16: **if** $bestOp = -t_\delta$ **then** ▷ Mask the deleted tuple
17: $Mask[\delta] = 1$
18: $\Delta D.append(bestOp);$
19: **return** $\Delta D;$

Lines 2 and 4 of the algorithm handle the initialization of variables. Lines 3-17 employ a greedy approach to sequentially select operations that maximize the current Qerror. Specifically, lines 5-13 iterate through the modified relation table one by one, computing the weights for both deletion and duplicate insertion for each tuple. In each iteration, the algorithm greedily selects the operation that maximizes the local Qerror. Lines 16-17 temporarily save the deleted tuples to prevent duplicate computation on already deleted tuples.

The aforementioned algorithm guarantees termination within polynomial-time. Specifically, the time complexity of the algorithm is $O(|\mathcal{R}es| + N \times K \times M)$, where: $|\mathcal{R}es|$ is the overhead of the group by operation in line 1 and is linear to the size of the result set $\mathcal{R}es$, N is the cardinality of the relation R_x, K is the budget allocated to the auditor for data insertion, and M is the size of the test query.

4.3 Analysis on the Perturbation Strategy

This section evaluates the perturbation strategy in Algorithm 1. We show it achieves optimal or near-optimal results when operations are limited to either insertions or deletions. Furthermore, we provide two corollaries for general scenarios and prove that the algorithm maintains a $1-\kappa$ approximation ratio in deletion-only cases.

Theorem 4.3. *In the scenario where only deletions are considered, let $\mathbb{Q}(O)$ denote the optimal DSA result. Algorithm 1 can achieve an approximation of $(1-\kappa)$. This is, the output ΔD of the Algorithm 1 possesses the property that $\mathbb{Q}(\Delta D) \geq (1-\kappa)\mathbb{Q}(O)$, where $\kappa = 1 - \min_{t \in R_x} \min_{A,B \subseteq R_x - t} \frac{\mathbb{Q}(A+t)-\mathbb{Q}(A)}{\mathbb{Q}(B+t)-\mathbb{Q}(B)}$.*

Proof. (Sketch).

Based on the definition of κ, we obtain $\forall A, B \subseteq R_x, 1-\kappa \leq \min_{t \in R_x} \frac{\mathbb{Q}(A+t)-\mathbb{Q}(A)}{\mathbb{Q}(B+t)-\mathbb{Q}(B)}$. Furthermore, we let the set ΔD_i denote the set chosen by Algorithm 1 at the i-th step, and we let ΔD_i to substitute A. Similarly, we can arbitrarily choose a subset permutation of the optimal solution O and let a subset O_i with size i from the permutation to substitute B. We also define $t_{i1} = O_{i+1} - O_i$ and $t_{i2} = \Delta D_{i+1} - \Delta D_i$. By definition, we can derive that:

$$1 - \kappa \leq \frac{\mathbb{Q}(t_{i1} + \Delta D_i) - \mathbb{Q}(\Delta D_i)}{\mathbb{Q}(t_{i1} + O_i) - \mathbb{Q}(O_i)}$$

Thus, we obtain:

$$(\mathbb{Q}(t_{i1} + O_i) - \mathbb{Q}(O_i)) \times (1-\kappa) \leq \mathbb{Q}(t_{i1} + \Delta D_i) - \mathbb{Q}(\Delta D_i) \quad (5)$$

Given that the outer loop of the Algorithm 1 runs for K steps, we can apply the aforementioned operation at each step of the process. Consequently, we have:

$$(1-\kappa)\mathbb{Q}(O) = (1-\kappa)\mathbb{Q}(\emptyset) + (1-\kappa)\sum_{i=1}^{K} \mathbb{Q}(O_i + t_{i1}) - \mathbb{Q}(O_i).$$

Since $\mathbb{Q}(\emptyset) \geq 0$ and combining Eq. 5, we can conclude:

$$(1-\kappa)\mathbb{Q}(O) \leq \mathbb{Q}(\emptyset) + \sum_{i=1}^{K} \mathbb{Q}(t_{i1} + \Delta D_i) - \mathbb{Q}(\Delta D_i)$$

Finally, based on lines 6-9 of Algorithm 1 at each step, we have $(\mathbb{Q}(t_{i2} + \Delta D_i) - \mathbb{Q}(\Delta D_i)) \geq (\mathbb{Q}(t_{i1} + \Delta D_i) - \mathbb{Q}(\Delta D_i))$. Therefore:

$$\begin{aligned}(1-\kappa)\mathbb{Q}(O) &\leq \mathbb{Q}(\emptyset) + \sum_{i=1}^{K} (\mathbb{Q}(t_{i1} + \Delta D_i) - \mathbb{Q}(\Delta D_i)) \\ &\leq \mathbb{Q}(\emptyset) + \sum_{i=1}^{K} (\mathbb{Q}(t_{i2} + \Delta D_i) - \mathbb{Q}(\Delta D_i)) \\ &= \mathbb{Q}(\Delta D_K).\end{aligned}$$

This means that:

$$(1-\kappa) \times \mathbb{Q}(O) \leq \mathbb{Q}(\Delta D_K).$$

Thus, we prove that the approximation ratio of algorithm is $1-\kappa$. □

After considering the scenario where only deletions are made, we will continue to analyze our algorithm's performance in the context of insertion-only scenarios. Fortunately, we conclude that, when considering only insertions, our algorithm is capable of achieving the optimal solution constituted solely by insertions.

Theorem 4.4. *In the case of considering only insertions, the above greedy approach can reach an optimal insertion result.*

Proof. (Sketch). **Base Case ($K = 1$):** When $K = 1$, the algorithm iterates through all possible values and identifies the tuple that maximizes the gain function G for insertion. Therefore, the algorithm is optimal at this initial step. **Inductive Step ($K \geq 2$):** Assume that for $K-1$, the algorithm achieves optimality. We need to show that under this assumption, the algorithm also achieves optimality for K. Suppose that ΔD_{K-1} achieves the maximal Qerror and inserts $K-1$ tuples. Let the optimal insertion perturbation strategy select tuple t_a at the K-th step, while our Algorithm 1 selects t_b. According to lines 10-13 of Algorithm 1, the following inequality holds:

$$\sum_{j=1}^{M} \frac{w_{bj}}{C_D(\mathbf{q}_j) + 1} \geq \sum_{j=1}^{M} \frac{w_{aj}}{C_D(\mathbf{q}_j) + 1}$$

This implies that $\mathbb{Q}'(t_b + \Delta D_{K-1}) \geq \mathbb{Q}'(t_a + \Delta D_{K-1})$. Thus, if that optimality is achieved at step $K-1$, using lines 10-13 of the Algorithm 1 ensures that the algorithm remains optimal for step K. □

Although the two aforementioned theorems are only preliminary considering insert-only or delete-only optimal DSA scenarios. We will now demonstrate, through the following corollaries, that in certain special cases, even when considering both insertions and deletions simultaneously, our algorithm can still achieve good approximation performance. The proof of these corollaries can be found in the appendix [7].

Corollary1: If the auditor can pre-determine whether each query in the testing workload is insertion-dominated or deletion-dominated, this mathematically removes the inner maximization layer of the Qerror summation in Eq. 3. Under this condition, ΔD of Algorithm 1 retains the following property: $\mathbb{Q}(\Delta D) \geq (1-\kappa)\mathbb{Q}(O)$.

Corollary 2: If the queries meet the following conditions: $M > K$, and for all queries $\mathbf{q}_j$, we have $\text{count}(w_{i,j} \neq 0) \leq 2$ and $C_D(\mathbf{q}_j) > K \times M$. Then, Algorithm 1 still possesses the following property: $\mathbb{Q}(\Delta D) \geq (1-\kappa)\mathbb{Q}(O)$.

The theorem and corollary demonstrate that while computing the optimal DSA is NP-Hard, Algorithm 1 offers a polynomial-time $(1-\kappa)$-approximation strategy under practical constraints. The analysis presented allows us to approximate the worst case of learned estimators within polynomial-time, prior to the occurrence of catastrophic estimation errors. This insight enables database administrators to effectively quantify potential weaknesses in estimators, providing a foundation for robust query optimization strategies.

5 EXPERIMENT

In this section, we provide an experimental analysis of the robustness limits of estimators under near worst-case data drifts. We will use our experiments to answer the following questions:

1. **Robustness Quantification:** Can the proposed DSA techniques effectively evaluate the robustness limits of data-driven, query-driven, and hybrid estimators? Do these estimators exhibit high sensitivity to small, targeted data modifications? (§ 5.3)
2. **Scalability:** How do variations in data size, query scale, and the selection of perturbed tables influence the efficiency and effectiveness of the proposed DSA technique? (§ 5.4)
3. **Consequence & Defense:** How would the degraded estimator mislead the query optimizer into selecting suboptimal query plans? What are the characteristics of such plans? Do we have countermeasures to mitigate the impact of DSA and enhance the worst-case performance for learned estimators? (§ 5.5)

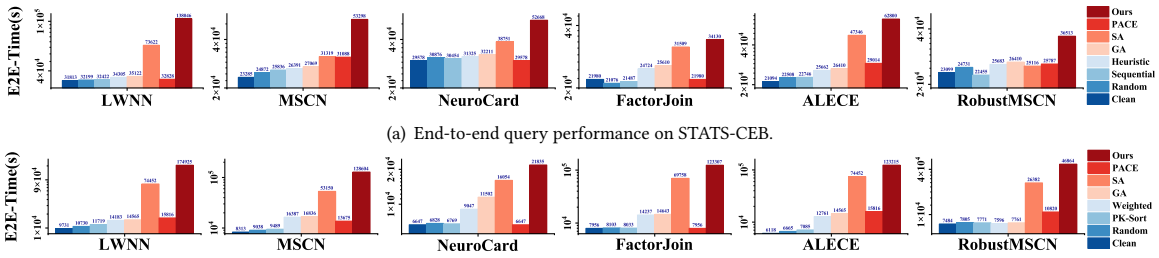
5.1 Experimental Setup

Datasets: We evaluate sensitivity analysis in multi-table settings using two widely adopted benchmarks, IMDB-JOB and STATS-CEB, which are commonly used for multi-table CE evaluation [30, 38, 47, 50, 59, 67]. IMDB-JOB is based on the IMDB database [1] and contains six relations with 14 attributes and 62,118,470 tuples [44]. We use the open-source JOB-light workload [3], which includes 70 queries and 696 subqueries. STATS-CEB is derived from anonymized StackExchange data [2, 30] and contains eight relations with 43 attributes and 1,029,842 tuples. We use the open-source workload [4] with 146 queries and 2,603 subqueries. Following prior protocols [4, 47], we normalize each query (and optimizer sub-expressions) into a unified cardinality-estimation form `SELECT COUNT(*)` over the same FROM/JOIN/WHERE clause; thus W_{Test} consists of SPJ subqueries without output aggregates (e.g., `MIN`) [44]. To train query-driven estimators, we follow Li et al. [47] and generate 2,000 training queries along with their subqueries.

CE models: Based on various studies [47, 50, 59], we selected the following most competitive and representative learned estimators covering the paradigms of data-driven, query-driven, and hybrid:
(1) *LWNN (Query-driven)* [26]: uses MLPs to map query representations to cardinalities. We follow the instructions in [30] to extend LWNN to support joins.
(2) *MSCN (Query-driven)* [39]: the most famous learned query-driven method based on a multiset convolutional network model.
(3) *NeuroCard (Data-driven)* [64]: learns a single deep auto-regressive model for the joint distribution of all tables in the database and estimates the cardinalities using the learned distribution. Following the settings in [64], we set the sampling size of the NeuroCard to 8,000.
(4) *Factorjoin (Data-driven)* [59]: integrates classical join-histogram methods and learned Bayes Network into a factor graph.
(5) *ALECE (Hybrid)* [47]: uses a transformer to learn the mapping from query representation and histogram features to cardinality.
(6) *RobustMSCN (Hybrid)* [50]: is an improvement to the basic MSCN method [39], aims at improving the robustness of the naive MSCN by employing masked training and incorporating data-leveled features such as PG estimates and join bitmaps.
(7) *Oracle*: is the oracle E_O which is capable of utilizing all information from the contaminated database D' to obtain query $\mathbf{q}$'s true cardinality $C_{D'}(\mathbf{q})$ within D'.

Baselines: We compare DSA with the estimators before any perturbations (Clean). Additionally, we compare our approach against the following baselines, selected for their relevance in reducing performance overheads in learned estimators or their application in analogous NP-Hard database optimization tasks [13, 33, 47, 50, 69]:
(1) *PACE* [69]: is the State-Of-The-Art query-centric analysis method on learned query-driven estimators. PACE tests query-driven estimators by contaminating historical workloads.
(2) *Random Generation (Random)*: randomly selects tuples from the training database to delete or duplicate.
(3) *Sequential Sort (Sequential)*: Simulates natural temporal drift by modifying data sorted by primary key, adopting the standard sequential update strategy used in robustness analysis [47, 50].
(4) *Greedy (Heuristic)*: Greedily removes tuples with the highest joint weights to investigate if fragility arises simply from the loss of "heavy-hitter" data.
(5) *Simulated Annealing (SA)* [14]: constructs perturbation strategies via a probabilistic hill-climbing algorithm and maximizes Eq. 3.
(6) *Genetic Algorithm (GA)* [49]: treats a set of potential perturbation strategies as a gene, by randomly selecting several sets of possible strategies and using them as a population for genetic crossover and mutation, therefore maximizing Eq. 3.

Experimental Settings: *Estimators Settings*: For the cardinality estimators to be tested, we train them on the contaminated database D' and test them on the clean database D. This choice intentionally isolates the estimator-induced errors in plan selection from confounding factors such as changed table sizes or changed join selectivities at execution time: the runtime and true cardinalities are always measured on the same clean database D, while only the estimator's internal "belief" is shifted by training on D'. That is to say, the cardinality labels for the Qerror evaluation and the data used to evaluate the actual latency are both derived from the clean database



(a) End-to-end query performance on STATS-CEB.

(b) End-to-end query performance on IMDB-JOB.

Figure 4: End-to-end(E2E) latency degradation per query on STATS/IMDB database.

D. For hybrid methods, we provide the data features of the clean database during testing. Specifically, we supply ALECE with histogram representations from the clean database D, and provide RobustMSCN with PG estimates and joint sampling features from the clean database. *Stress Test Settings*: In § 5.3, we target the `title` table of the IMDB-JOB database and the `posts` table of the STATS-CEB database.Guided by the budget calibration in § 5.2, we fix the maximum perturbation budget at 20% of the targeted table. We emphasize that this threshold is selected to ensure the sufficient "activation" of baseline methods for effective performance stratification, rather than a necessity for DSA, which exposes combinatorial vulnerabilities at much lower budgets (e.g., 1.25% in table 1 Consequently, our setting circumvents the massive global drifts (> 20%) typically required by prior studies (e.g., ALECE [47]), achieving rigorous stress testing with a global modification scale that is an order of magnitude lower. Specifically, the auditor modifies only $\frac{5\times10^5}{6.2\times10^7} \approx 0.8\%$ of the total records in IMDB-JOB, and $\frac{1.8\times10^4}{1\times10^6} \approx 1.2\%$ in STATS-CEB.

Metrics: We use the metrics defined in § 2.3 to evaluate the stress test's effectiveness. We use Qerror to measure the degradation in estimation accuracy to evaluate the quality of the generated query plans, and end-to-end (E2E) time to assess the impact of estimation on the end-to-end execution of queries. We utilize the framework developed in [30] to inject the cardinalities predicted by the estimator into Postgres and evaluate the end-to-end execution time.

Environments: Our end-to-end experiments were conducted on an individual Huawei Cloud server with 4 Intel(R) Xeon(R) Platinum 8378A CPUs and 32GB RAM and 1T SSD. Apart from that, all remaining experiments were run on a server with 4 RTX A6000 GPUs, 40 Intel(R) Xeon(R) Silver 4210R CPUs, and 504GB RAM.

5.2 Robustness Sensitivity & Budget Calibration

To establish a perturbation budget that is minimal yet sufficient to distinguish the efficacy of different drift patterns, we have to first isolate errors from model-specific biases. To this end, we employ the Oracle estimator (E_O) as a probe. By granting E_O perfect knowledge of the perturbed database state D', we ensure that any performance degradation on the current database D is attributable solely to the distribution shift ΔD, eliminating confounding factors such as binning errors or neural approximation limits. In this controlled environment, we benchmark DSA against the Sequential and Heuristic on STATS (with extended sensitivity analyses provided in Appendix). This evaluation yields two critical insights:

First, robustness under limited budgets is inherently a combinatorial problem. As shown in Table 1, at a negligible budget of 1.25% of the target table (only 0.11% of the total database), heuristic baselines fail to induce meaningful degradation (Median Q-Error ≈ 1.06), contradicting the assumption that simply removing "heavy-hitters" suffices to compromise the estimator. In contrast, DSA exposes structural fragility by identifying a worst-case subset that drives the 95th percentile error to 396 and the maximum error to 1.64×10^6 under the same budget. Second, fair comparison requires a budget calibrated to "activate" baseline methods. Contrary to prior work suggesting that large global drifts (> 20% of the *entire* database) are necessary for performance decay [47, 50], our results show that baselines already exhibit observable degradation when 20% of the target table is modified (≈ 1.79% of the total database), where the Heuristic approach reaches a Median Q-Error of 3.20. Although DSA saturates at the much lower 0.11% global level, we standardize the perturbation budget at this 20% target-level threshold (≈ 1.79% global) for subsequent experiments in Section 5.3, ensuring both a rigorous evaluation space for heuristic methods and evidence that worst-case analysis does not require massive global drifts.

Takeaways: DSA breaches reaches CE boundaries at negligible data drifts (0.11% of total DB) where heuristics remain silent. For fair comparison, we adopt a conservative 20% budget (≈ 1.79% of total DB) to activate baselines, enabling effective performance stratification without massive updates.

Table 1: Calibration on Oracle (E_O) on STATS-CEB.

Drift Constraint / Budget			Audit Method	Sensitivity Metric (Q-Error)			
Rows	Table %	Total DB %		50%	90%	95%	Max
1,150	1.25%	0.11%	Sequential	1.00	1.04	1.06	1.15
			Heuristic	1.06	1.67	2.74	58.4
			DSA (Ours)	**1.40**	**16.5**	**396**	1.64×10^6
2,299	2.50%	0.22%	Sequential	1.02	1.09	1.15	1.25
			Heuristic	1.10	2.11	3.34	78.7
			DSA (Ours)	**1.55**	**146**	4.19×10^3	1.64×10^6
4,599	5.00%	0.45%	Sequential	1.05	1.23	1.43	1.94
			Heuristic	1.27	5.65	9.17	137
			DSA (Ours)	**1.81**	1.62×10^6	1.02×10^8	1.11×10^9
9,198	10.0%	0.89%	Sequential	1.13	1.38	1.66	2.24
			Heuristic	1.94	15.0	22.6	1.39×10^3
			DSA (Ours)	**9.82**	4.15×10^7	1.47×10^8	1.78×10^{10}
18,395	20.0%	1.79%	Sequential	1.38	1.97	2.25	4.09
			Heuristic	3.20	56.2	110	4.75×10^3
			DSA (Ours)	1.67×10^4	1.00×10^8	2.50×10^8	1.78×10^{10}

5.3 Decline of the Estimator's Performance

Q-error Distribution Analysis: We evaluate estimator robustness across the Q-error distribution (Table 2), as the median reflects general-purpose utility [65] while tail errors govern the risk of catastrophic query optimization [66, 69]. Our approach consistently dominates all baselines across query-driven, data-driven, and hybrid architectures. For query-driven models (e.g., LWNN, MSCN),

Table 2: Percentile Qerrors on STATS-CEB and IMDB-JOB.

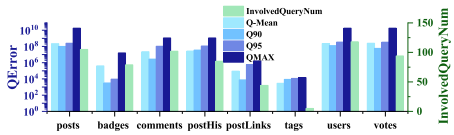
PARADIGM	CE METHOD	PERTURBATION METHOD	IMDB-JOB QERROR					STATS-CEB QERROR				
			50%	90%	95%	99%	MAX	50%	90%	95%	99%	MAX
Query-Driven	LWNN	Clean	2.661	65.04	117.6	1567	$5.67 \cdot 10^4$	4.762	25.98	47.18	1182	$2.66 \cdot 10^4$
		Random	2.271	64.52	118.2	1595	$3.81 \cdot 10^4$	4.489	20.37	32.10	1505	$3.51 \cdot 10^4$
		Sequential	3.675	81.12	228.2	9489	$3.84 \cdot 10^4$	5.085	44.12	143.0	1416	$3.86 \cdot 10^4$
		Heuristic	13.9	$3.18 \cdot 10^3$	$1.40 \cdot 10^4$	$1.40 \cdot 10^5$	$2.57 \cdot 10^5$	5.44	85	601	$3.73 \cdot 10^4$	$4.03 \cdot 10^5$
		GA	48.61	1105	5570	$8.41 \cdot 10^4$	$9.60 \cdot 10^4$	21.84	377.7	1167	9173	$5.52 \cdot 10^4$
		SA	184.1	$2.05 \cdot 10^4$	$4.21 \cdot 10^4$	$1.28 \cdot 10^5$	$1.91 \cdot 10^5$	39.87	1065	3847	$4.65 \cdot 10^4$	$3.67 \cdot 10^6$
		PACE	27.54	1359	6542	$4.79 \cdot 10^4$	$9.59 \cdot 10^4$	26.44	654.8	2486	$1.36 \cdot 10^4$	$7.23 \cdot 10^7$
		Ours	**350.3**	$\mathbf{3.29 \cdot 10^4}$	$\mathbf{9.12 \cdot 10^4}$	$\mathbf{1.51 \cdot 10^5}$	$\mathbf{9.35 \cdot 10^8}$	**532.8**	$\mathbf{2.43 \cdot 10^4}$	$\mathbf{8.35 \cdot 10^4}$	$\mathbf{1.14 \cdot 10^6}$	$\mathbf{1.14 \cdot 10^8}$
	MSCN	Clean	2.076	13.92	53.68	199.3	1653	2.235	99.10	179.1	505.1	2689
		Random	2.085	20.14	59.14	178.1	2116	2.724	161.8	298.8	732.2	4342
		Sequential	3.106	28.77	104.5	494	$1.92 \cdot 10^5$	3.60	180.3	410.5	4610	9812
		Heuristic	9.55	402	$2.64 \cdot 10^3$	$5.05 \cdot 10^4$	$1.40 \cdot 10^5$	4.32	280	355	$2.28 \cdot 10^4$	$6.88 \cdot 10^4$
		GA	18.58	443.2	2111	$2.13 \cdot 10^4$	$1.12 \cdot 10^6$	19.71	397.2	1257	$1.09 \cdot 10^4$	$5.03 \cdot 10^4$
		SA	30.07	2198	6911	$2.51 \cdot 10^4$	$1.71 \cdot 10^5$	38.31	3731	$1.58 \cdot 10^4$	$1.05 \cdot 10^5$	$1.24 \cdot 10^5$
		PACE	54.60	569.1	1437	$4.46 \cdot 10^4$	$2.11 \cdot 10^5$	10.01	117.6	209.2	1488	9724
		Ours	**60.23**	**6201**	$\mathbf{3.04 \cdot 10^4}$	$\mathbf{1.56 \cdot 10^5}$	$1.46 \cdot 10^6$	**184.3**	$4.63 \cdot 10^6$	$1.16 \cdot 10^7$	$4.74 \cdot 10^7$	$8.08 \cdot 10^8$
Data-Driven	NeuroCard	Clean	1.416	3.312	8.32	18.39	21.51	2.621	1089	6643	$4.95 \cdot 10^4$	$1.71 \cdot 10^6$
		Random	1.869	4.632	8.533	25.97	27.03	2.986	1092	6426	$4.95 \cdot 10^4$	$4.85 \cdot 10^6$
		Sequential	1.876	5.47	9.4	32	43.03	3.18	1351	$4.32 \cdot 10^3$	$8.95 \cdot 10^4$	$2.23 \cdot 10^6$
		Heuristic	4.7	91.8	181	768	$4.93 \cdot 10^3$	3.38	$1.47 \cdot 10^3$	$3.03 \cdot 10^3$	$4.27 \cdot 10^5$	$4.33 \cdot 10^6$
		GA	1.884	11.72	56.56	830.9	971.6	4.09	1073	6690	$6.6 \cdot 10^5$	$1.04 \cdot 10^6$
		SA	39.58	484	1031	3626	6618	3.967	1181	7211	$4.95 \cdot 10^5$	$1.04 \cdot 10^7$
		PACE	1.416	3.312	8.32	18.39	21.51	2.621	1089	6643	$4.95 \cdot 10^4$	$1.71 \cdot 10^6$
		Ours	**108.4**	**2972**	**3888**	$\mathbf{1.53 \cdot 10^4}$	$\mathbf{3.47 \cdot 10^4}$	**15.91**	$\mathbf{1.8 \cdot 10^5}$	$\mathbf{1.12 \cdot 10^6}$	$\mathbf{4.09 \cdot 10^7}$	$\mathbf{2.39 \cdot 10^8}$
	FactorJoin	Clean	3.015	6.82	39.11	$1.36 \cdot 10^4$	$1.98 \cdot 10^5$	2.018	40.61	78.91	302.7	760.1
		Random	3.176	5.481	45.82	$1.63 \cdot 10^4$	$8.01 \cdot 10^5$	2.072	45.98	64.56	312.98	774.3
		Sequential	3.145	16.21	101.0	$3.25 \cdot 10^4$	$3.64 \cdot 10^5$	2.19	35.7	60.2	455	875
		Heuristic	4.11	312	$3.29 \cdot 10^4$	$8.90 \cdot 10^4$	$1.10 \cdot 10^7$	2.46	218	611	$4.32 \cdot 10^4$	$3.80 \cdot 10^5$
		GA	4.088	156.7	4087	$1.19 \cdot 10^5$	$1.54 \cdot 10^6$	5.03	115.8	296.2	921.8	$1.13 \cdot 10^4$
		SA	3.999	889.6	$1.76 \cdot 10^4$	$8.76 \cdot 10^5$	$3.68 \cdot 10^8$	6.18	784.6	$1.67 \cdot 10^4$	$2.22 \cdot 10^7$	$1.41 \cdot 10^9$
		PACE	3.015	6.82	39.11	$1.36 \cdot 10^4$	$1.98 \cdot 10^5$	2.018	40.61	78.91	302.7	760.1
		Ours	**6.706**	$\mathbf{1.56 \cdot 10^5}$	$\mathbf{2.33 \cdot 10^6}$	$\mathbf{4.523 \cdot 10^7}$	$\mathbf{1.04 \cdot 10^9}$	**6.486**	**2099**	$\mathbf{8.05 \cdot 10^4}$	$\mathbf{1.72 \cdot 10^8}$	$\mathbf{4.52 \cdot 10^9}$
Hybrid	ALECE	Clean	1.374	2.611	3.702	16.03	649.4	1.532	3.007	4.321	19.55	48.89
		Random	1.566	2.741	3.499	9.782	439.7	1.648	3.563	4.583	27.04	68.4
		Sequential	1.62	4.8	3.77	12.5	558.6	1.82	4.28	5.84	43.8	92.7
		Heuristic	9.21	153	$1.16 \cdot 10^3$	$9.48 \cdot 10^3$	$8.64 \cdot 10^4$	1.64	56.4	61.2	583	$1.83 \cdot 10^3$
		GA	13.86	159.2	212.9	5547	$1.28 \cdot 10^5$	4.877	76.31	186.9	417.5	2027
		SA	27.01	696.1	2723	$5.06 \cdot 10^4$	$1.49 \cdot 10^5$	4.32	724.5	1405	3458	4631
		PACE	7.229	134.4	265.6	4284	$4.62 \cdot 10^5$	4.08	207.5	544.8	5637	$3.03 \cdot 10^4$
		Ours	**29.33**	**9323**	$\mathbf{2.73 \cdot 10^4}$	$\mathbf{2.45 \cdot 10^5}$	$\mathbf{2.02 \cdot 10^6}$	**6.442**	$\mathbf{3.23 \cdot 10^4}$	$\mathbf{2.06 \cdot 10^5}$	$\mathbf{5.88 \cdot 10^6}$	$\mathbf{1.64 \cdot 10^7}$
	RobustMSCN	Clean	1.811	4.415	23.67	355.5	1102	2.288	5.196	7.43	28.62	4470
		Random	2.145	5.409	23.81	381.5	2547	2.723	5.903	7.743	23.55	4656
		Sequential	2.19	7.45	24.4	406	$2.21 \cdot 10^3$	2.73	7.62	9.97	41.7	$7.20 \cdot 10^3$
		Heuristic	5.03	81.6	113	$1.34 \cdot 10^3$	$2.02 \cdot 10^4$	2.9	26.8	35.4	325	$3.22 \cdot 10^4$
		GA	15.30	109.1	173.8	627.8	$1.83 \cdot 10^4$	3.820	47.62	81.41	274.9	$7.38 \cdot 10^4$
		SA	12.69	186.3	805.5	5705	$2.16 \cdot 10^4$	3.92	83.2	270.5	1225	$2.85 \cdot 10^4$
		PACE	5.019	77.26	278.7	2352	7155	4.84	98.18	426.7	4009	$1.48 \cdot 10^4$
		Ours	**15.87**	**814.7**	**2185**	$\mathbf{1.52 \cdot 10^4}$	$\mathbf{2.64 \cdot 10^5}$	**5.19**	**2388**	**8762**	$\mathbf{1.32 \cdot 10^5}$	$\mathbf{7.06 \cdot 10^5}$
Oracle		Clean	1	1	1	1	1	1	1	1	1	1
		Random	1.255	1.294	1.663	1.766	3.338	1.212	1.298	1.319	1.515	2.328
		Sequential	2.15	2.44	2.77	244.8	780.2	1.375	1.969	2.250	3.763	4.092
		Heuristic	18.31	$1.86 \cdot 10^3$	$1.85 \cdot 10^4$	$1.15 \cdot 10^6$	$3.05 \cdot 10^6$	3.200	56.20	110.0	1519	4749
		GA	83.39	230.3	2005	$5.15 \cdot 10^5$	$1.674 \cdot 10^6$	25.97	258.8	1566	8741	$1.44 \cdot 10^4$
		SA	301.1	$2.72 \cdot 10^5$	$5.12 \cdot 10^5$	$1.54 \cdot 10^7$	$3.23 \cdot 10^7$	35.98	2133	4835	$1.43 \cdot 10^4$	$1.56 \cdot 10^6$
		Ours	$\mathbf{1.49 \cdot 10^5}$	$\mathbf{1.23 \cdot 10^8}$	$\mathbf{5.03 \cdot 10^8}$	$\mathbf{3.89 \cdot 10^9}$	$\mathbf{9.53 \cdot 10^9}$	$\mathbf{1.66 \cdot 10^4}$	$\mathbf{1.02 \cdot 10^8}$	$\mathbf{2.50 \cdot 10^8}$	$\mathbf{2.21 \cdot 10^9}$	$\mathbf{1.78 \cdot 10^{10}}$

we outperform competitors (SA, PACE, GA, Random) by factors ranging from 6.85× to 1876×. Crucially, the stark contrast between our method and the "Random" baseline reveals a fundamental vulnerability: while modern estimators are resilient to stochastic drift, they are strikingly fragile under targeted modifications. This confirms that robustness is distribution-dependent rather than absolute. Specifically, our approach achieves 2.43×–47.1× degradation at the 50th percentile. This efficacy extends to data-driven models (e.g., NeuroCard), where PACE [69] often fails because it targets historical workloads; in contrast, our strategy perturbs underlying data correlations to degrade even distribution-centric estimators. The most significant impact occurs at the tail (above the 90th percentile), where our method inflates errors by 3–4 orders of magnitude. Such extreme misestimates are fatal for join-order selection, resulting in catastrophic latency. Furthermore, we observe a "perfection paradox": high-accuracy estimators that closely fit the training oracle E_O (e.g., NeuroCard on IMDB) are structurally vulnerable, suffering median error increases of two orders of magnitude under our stress tests. These results validate the near-optimality of Algorithm 1 (§ 4.3) in navigating the NP-Hard search space and confirm the theoretical transformations in § 3.2.

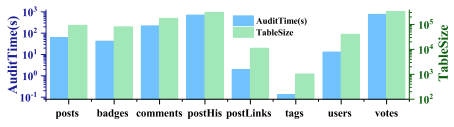
Decline of the E2E-Time: To assess how different perturbation strategies affect query optimization, we measured end-to-end execution time across multiple estimators. As shown in Figures 4(a) and 4(b), our method consistently produces the worst execution plans and the largest slowdowns. On IMDB-JOB, it increases execution time by 24.2×, 2.21×, 17.5×, and 202× over PACE, SA, GA, and Random, respectively; on STATS-CEB, the increases reach 11.6×, 2.34×, 10.3×, and 41.8×. Estimators exhibit varying vulnerability to data-centric sensitivity analysis (LWNN > MSCN > NeuroCard > ALECE > FactorJoin > RobustMSCN), with RobustMSCN showing relative resilience but still slowing by 1.58×. Overall, these results confirm that our sensitivity analysis exposes estimator fragility and drives optimizers toward significantly suboptimal plans.

Takeaways: Our DSA framework systematically uncovers vulnerabilities across data-driven, query-driven, and hybrid estimators. Moving beyond simple heuristics, our global optimization identifies strategic data subsets that inflict disproportionate damage. DSA significantly degrades worst-case accuracy, forcing suboptimal execution plans and excessive latency. Notably, even hybrid models offer only certain robustness against such targeted perturbations.

5.4 Scalability Study



(a) Perturbation Effectiveness on different tables.



(b) Perturbation Efficiency on different tables.

Figure 5: Scalability on different tables.

Scalability on Perturbed Tables: To validate the theoretical perturbation effectiveness on different tables, in Figure 5(a) and Figure 5(b), we conducted experiments by switching the targeted tables within the STATS-CEB database. We selected the oracle E_O as the target. The perturbation budget was set at 20% of the rows in the targeted table. Our findings indicate that, despite the varying number of queries associated with different tables in the STATS-CEB workload, our method can significantly impact E_O for any table, resulting in a maximum Qerror ranging from 10^4 to 10^{10}. Additionally, we observed that the more queries a table encompasses, the greater the impact of the perturbation. For instance, when the number of queries increased from 10 (in `tags`) to 76 (in the `votes`), the maximum Qerror escalated from 10^4 to 10^{10}.

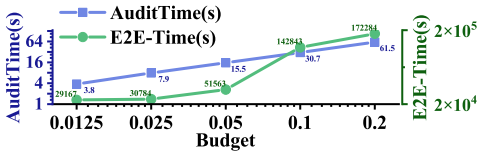


Figure 6: Scalability on different budget.

Scalability w.r.t Perturbation Budget. We evaluate the execution time of DSA on the E_O estimator while increasing the perturbation budget from 1.25% to 20% on the `posts` table. As illustrated in Figure 6, the audit time grows linearly with the budget size K. Even at the maximum tested budget of 20% (where estimator performance is completely degraded), the analysis completes in acceptable limits.

Takeaways: The overhead of our proposed DSA analysis strategy is linear to the perturbation budget and correlated with table size and query number. Moreover, the more queries that involve the targeted table, the larger the perturbation budget, and consequently, the greater the benefit derived from the sensitivity analysis.

5.5 Utility: Auditing, Guidance, and Diagnostics

In this section, we explore the utility of our DSA. Based on the worst-case discovered by DSA, we devised a spectrum of five distinct approaches to investigate and mitigate worst-case failures. We detail three primary applications (Maintenance, Architecture, and Diagnostics) in the main text, while extending our analysis to two further mechanisms to boost the worst-case performance in our Appendix.

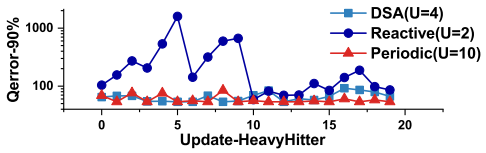
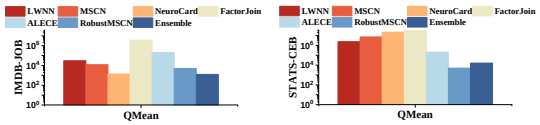


Figure 7: Utility: DSA as an Early Warning Sentinel.

Application I: Robust Model Maintenance Auditing: A critical application of DSA is determining when to update learned models in dynamic environments. In real-world scenarios with "Heavy Hitter" updates, fixed strategies often fail. To simulate this, we utilized the STATS-CEB-post table and the LWNN estimator, performing 20 rounds of updates where each round modifies 1% of the data (detailed settings are provided in the Appendix). As shown in Figure 7, the Reactive (Post-hoc) strategy triggers retraining only after detecting high errors; consequently, it incurs unavoidable production losses, evidenced by catastrophic Q-error spikes (> 1000) before mitigation occurs. The Periodic (High Budget) strategy ensures stability by blindly retraining every two rounds ($U = 10$) but suffers from prohibitive computational overhead. In contrast, our DSA-Guided Sentinel proactively identifies critical tuple shifts. It achieves stability comparable to the periodic method with only four effective updates ($U = 4$), saving 60% of the budget. This confirms DSA's utility as a cost-effective sentinel for data drift.

Application II: Guidance for Robust System Design Beyond determining maintenance timing, the value of our DSA also lies in guiding the architectural design of more robust estimators. We investigated whether architectural redundancy enhances robustness. As shown in Figure 8, we tested an ensemble strategy guided by sensitivity analysis, which performs a weighted fusion of six different estimators. While this ensemble approach mitigates performance degradation in worst-case scenarios to some extent, significant room for optimization remains. To further explore architectural defenses, we investigate Hardening via Controlled Noise Injection in Appendix H. This strategy establishes a statistical "buffer zone" against distribution shifts and reveals a critical tradeoff between nominal precision and worst-case stability.

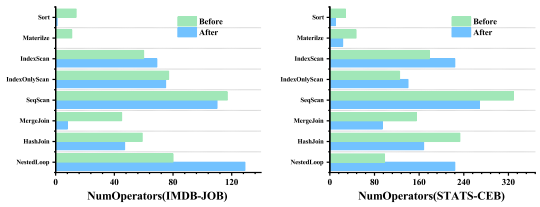


(a) Ensemble on: IMDB-JOB. (b) Ensemble on: STATS-CEB.

Figure 8: Robustness Assessment of Ensemble Architectures.

Application III: Diagnostics of Optimizer Behavior To understand how estimator worst-case failures propagate to the downstream query optimizer, we utilized DSA to simulate worst-case scenarios on IMDB-JOB and STATS-CEB using LWNN, diagnosing the specific vulnerabilities of the query optimizer (Figure 9).

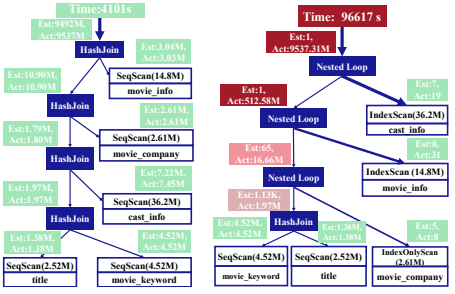
Vulnerability 1: Join Pattern Sensitivity. Our audit reveals that the optimizer is extremely sensitive to cardinality underestimation, leading to drastic shifts in join operators. As shown in Figure 9, under the pressure of DSA-generated updates, the usage of Nested Loop Joins surged by 128% (IMDB-JOB) and 61% (STATS-CEB), while efficient Hash Joins declined significantly. This indicates that when misled by worst-case estimates, the optimizer regresses to



(a) Distribution on: IMDB-JOB. (b) Distribution on: STATS-CEB.

Figure 9: Diagnostic: Operator Distribution Shift.

operators suitable only for small data, severely degrading complex analytical queries. A stark example is the regression of IMDB-JOB Q60 (Figure 10(a) vs. Figure 10(b)), where the optimizer abandoned Hash Joins for Index Nested Loops, causing a massive increase in latency. This finding prompts a natural heuristic defense: disabling risky operators. In Appendix G, we evaluate this "Safety-First" configuration (disabling NLJs). Our audit reveals that while this heuristic works for average cases, it fails at the tail, leading the optimizer into a "Merge Join Trap" (52× degradation). This persistent fragility underscores the inherent difficulty of the robustness problem, indicating that resolving these estimator blind spots requires structural advancements beyond simple operator constraints, necessitating broader community efforts.



(a) Plan before Audit. (b) Plan under Worst-Case Audit.

Figure 10: Case Study: Diagnosing Plan Regression on Q60.

Vulnerability 2: Cache Efficiency. The diagnostic also uncovered a hidden cost: reduced cache efficiency. We observed that materialized operators decreased by 90% in IMDB and 51% in STATS. This implies that worst-case errors not only affect current execution but also mislead the global plan into bypassing materialized views, triggering expensive redundant computations.

Takeaways: Our DSA offers five pathways to mitigate worst-case failures. We demonstrate: **(1) Proactive Maintenance** (saving 60% budget via "Sentinel" updates); **(2) Architectural Benchmarking** (testing ensembles); and **(3) Root Cause Diagnostics** (tracing optimizer faults). We further explore **(4) Operator Restrictions** and **(5) Noise Injection** defenses in the Appendix.

6 RELATED WORK

Learned Cardinality Estimators: Modern cardinality estimators can be categorized into three paradigms: query-driven [26, 39, 58], data-driven [48, 59, 64, 65, 65] and hybrid [47, 50]. (1) Query-driven estimators employ neural networks [26, 39, 58] to learn mappings from query representations to true cardinalities. Although query-driven methods can achieve quick and lightweight estimations, it is confirmed by many studies that query-driven estimators suffer from changes in testing query distribution, due to data drifts or shifts within query workloads [47, 55, 56, 65]. (2) Data-driven estimators learn joint data distributions and sample from deep models [38, 64, 65] to estimate cardinalities. They are robust against workload drifts [56, 65]. (3) Hybrid estimators [31, 39, 47, 50, 50, 58] combine data statistics with query features for unified learning. Commonly used data features include histograms [47, 67], sample collections [39, 50], and estimations from PostgreSQL [31, 50, 67].

Robustness of Learned Database Components: Nowadays, the robustness and stability of learned database components have garnered widespread attention [63, 63, 69, 71]. Kornaropoulos et al. [63, 63] introduced algorithmic complexity analysis targeting learned indexes by constructing adversarial inputs that cause learned indexes to fail. Additionally, Zheng et al. [71] proposed robustness evaluation aimed at learning-based index advisors. The work most closely related to ours is PACE proposed by Zhang et al. [69], which stress-tested query-driven estimators by adding noise to their training workloads. However, PACE has a clear limitation: it can only target query-driven models. In contrast, the sensitivity analysis framework proposed in this paper can evaluate the robustness limits of nearly all types of cardinality estimators.

Adversarial Data Drifts in ML: Traditional ML adversarial robustness is typically analyzed under the white-box assumption, where the auditor has full knowledge of the internal model [12, 15, 16, 34, 61, 62, 70]. For example: (1). Under the assumption of linear regression, Jagielski et al. [12] proposed an optimization framework for analyzing the sensitivity of linear regression models. (2). For support vector machines (SVMs) [15, 61, 62], adversarially crafted training data can manipulate the decision boundary, thereby increasing the error rate. (3). In the context of neural networks, Yang et al. [70] introduced a gradient-based technique to generate worst-case inputs. In contrast, our work focuses on a more realistic black-box scenario, where the auditor lacks knowledge of the internal model details. Specifically, the auditor is unaware whether the internal model is based on neural networks [47, 64], decision trees [26, 32, 75], Bayesian networks [59, 60], or other emerging models [55, 67].

7 CONCLUSION

This work provides the first theoretical investigation into the stability limits of learned cardinality estimators under minimal, targeted data drifts. We introduce a black-box Data-centric Sensitivity Analysis (DSA) framework, proving that while finding the optimal worst-case drift is NP-Hard, a polynomial-time $(1-\kappa)$-approximation is achievable. Experiments on STATS-CEB and IMDB-JOB benchmarks show our stress test's effectiveness: modifying merely 0.8% of database tuples causes a 1000× increase in 90-th percentile Qerror and up to 20× longer processing times. These findings expose the fragility of current estimators in real-world scenarios. We also propose practical countermeasures against such worst-case drifts, providing valuable insights for developing robust learned optimizers. Future work could focus on creating more resilient estimation techniques that can withstand adversarial data distributions while maintaining reliable performance.

REFERENCES

[1] [n.d.]. IMDB. http://homepages.cwi.nl/~boncz/job/imdb.tgz.,.

[2] [n.d.]. StackExchange. https://stats.stackexchange.com/,.

[3] 2021. Joblight Subqueries. https://github.com/Nathaniel-Han/End-to-End-CardEst-Benchmark/tree/master/workloads/job-light.

[4] 2021. STATS Subqueries. https://github.com/Nathaniel-Han/End-to-End-CardEst-Benchmark/tree/master/workloads/stats_CEB.

[5] 2023. ALECE: Evaluation Utilities. https://github.com/pfl-cs/ALECE/blob/main/src/utils/eval_utils.py

[6] 2023. PRICE: Evaluation Utilities. https://github.com/StCarmen/PRICE/blob/master/utils/model/qerror.py

[7] 2025. DACA. https://anonymous.4open.science/r/DACA-22F2/README.MD.

[8] Rana Alotaibi, Yuanyuan Tian, Stefan Grafberger, Jesús Camacho-Rodríguez, Nicolas Bruno, Brian Kroth, Sergiy Matusevych, Ashvin Agrawal, Mahesh Behera, Ashit Gosalia, Cesar Galindo-Legaria, Milind Joshi, Milan Potocnik, Beysim Sezgin, Xiaoyu Li, and Carlo Curino. 2025. Towards Query Optimizer as a Service (QOaaS) in a Unified LakeHouse Ecosystem: Can One QO Rule Them All?. In Proceedings of the 15th Conference on Innovative Data Systems Research (CIDR). https://www.cidrdb.org/cidr2025/papers/p5-alotaibi.pdf

[9] Yuichi Asahiro, Refael Hassin, and Kazuo Iwama. 2002. Complexity of finding dense subgraphs. Discrete Appl. Math. 121, 1–3 (Sept. 2002), 15–26. https://doi.org/10.1016/S0166-218X(01)00243-8

[10] Nirav Atre, Hugo Sadok, Erica Chiang, Weina Wang, and Justine Sherry. 2022. SurgeProtector: mitigating temporal algorithmic complexity attacks using adversarial scheduling. In Proceedings of the ACM SIGCOMM 2022 Conference (Amsterdam, Netherlands) (SIGCOMM '22). Association for Computing Machinery, New York, NY, USA, 723–738. https://doi.org/10.1145/3544216.3544250

[11] Wenruo Bai and Jeffrey A. Bilmes. 2018. Greed is Still Good: Maximizing Monotone Submodular+Supermodular (BP) Functions. In Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018 (Proceedings of Machine Learning Research), Jennifer G. Dy and Andreas Krause (Eds.), Vol. 80. PMLR, 314–323. http://proceedings.mlr.press/v80/bai18a.html

[12] Marco Barreno, Blaine Nelson, Anthony Joseph, and J. Tygar. 2010. The security of machine learning. Machine Learning 81 (11 2010), 121–148. https://doi.org/10.1007/s10994-010-5188-5

[13] Kristin P Bennett, Michael C Ferris, and Yannis E Ioannidis. 1991. A genetic algorithm for database query optimization. Technical Report. University of Wisconsin-Madison Department of Computer Sciences.

[14] Dimitris Bertsimas and John Tsitsiklis. 1993. Simulated annealing. Statistical science 8, 1 (1993), 10–15.

[15] Battista Biggio, Blaine Nelson, and Pavel Laskov. 2012. Poisoning attacks against support vector machines. In Proceedings of the 29th International Coference on International Conference on Machine Learning (Edinburgh, Scotland) (ICML'12). Omnipress, Madison, WI, USA, 1467–1474.

[16] Battista Biggio, Ignazio Pillai, Samuel Rota Bulò, Davide Ariu, Marcello Pelillo, and Fabio Roli. 2013. Is data clustering in adversarial settings secure?. In Proceedings of the 2013 ACM Workshop on Artificial Intelligence and Security (Berlin, Germany) (AISec '13). Association for Computing Machinery, New York, NY, USA, 87–98. https://doi.org/10.1145/2517312.2517321

[17] Mark Braverman, Young Kun Ko, Aviad Rubinstein, and Omri Weinstein. 2017. ETH hardness for densest-k-subgraph with perfect completeness. In Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms(SODA). SIAM, 1326–1341.

[18] Chandra Chekuri, Kent Quanrud, and Manuel R. Torres. [n.d.]. Densest Subgraph: Supermodularity, Iterative Peeling, and Flow. In Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA). 1531–1555. https://doi.org/10.1137/1.9781611977073.64 arXiv:https://epubs.siam.org/doi/pdf/10.1137/1.9781611977073.64

[19] Scott A. Crosby and Dan S. Wallach. 2003. Denial of service via algorithmic complexity attacks. In Proceedings of the 12th Conference on USENIX Security Symposium - Volume 12 (Washington, DC) (SSYM'03). USENIX Association, USA, 3.

[20] Carlo Curino, Evan Jones, Yang Zhang, and Sam Madden. 2010. Schism: a workload-driven approach to database replication and partitioning. Proc. VLDB Endow. 3, 1–2 (Sept. 2010), 48–57. https://doi.org/10.14778/1920841.1920853

[21] Kyle Deeds, Dan Suciu, Magdalena Balazinska, and Walter Cai. 2025. Degree Sequence Bounds. ACM Trans. Database Syst. 51, 1, Article 4 (Oct. 2025), 27 pages. https://doi.org/10.1145/3716378

[22] Kyle B. Deeds, Dan Suciu, and Magdalena Balazinska. 2023. SafeBound: A Practical System for Generating Cardinality Bounds. Proc. ACM Manag. Data 1, 1, Article 53 (May 2023), 26 pages. https://doi.org/10.1145/3588907

[23] Jialin Ding, Ryan Marcus, Andreas Kipf, Vikram Nathan, Aniruddha Nrusimha, Kapil Vaidya, Alexander van Renen, and Tim Kraska. 2022. SageDB: An Instance-Optimized Data Analytics System. Proc. VLDB Endow. 15, 13 (Sept. 2022), 4062–4078. https://doi.org/10.14778/3565838.3565857

[24] W Dong, Z Chen, Q Luo, E Shi, and K Yi. 2024. Continual Observation of Joins under Differential Privacy. In Proceedings of the ACM on Management of Data, Vol. 2. ACM, 1–27.

[25] W Dong, JY Fang, KT Yi, Y Tao, and A Machanavajjhala. 2024. Instance-optimal Truncation for Differentially Private Query Evaluation with Foreign Keys. In ACM Transactions on Database Systems, Vol. 49. ACM, 1–40.

[26] Anshuman Dutt, Chi Wang, Azade Nazi, Srikanth Kandula, Vivek Narasayya, and Surajit Chaudhuri. 2019. Selectivity estimation for range predicates using lightweight models. Proceedings of the VLDB Endowment (May 2019), 1044–1057. https://doi.org/10.14778/3329772.3329780

[27] Jan Finis, Robert Brunel, Alfons Kemper, Thomas Neumann, Norman May, and Franz Faerber. 2015. Indexing highly dynamic hierarchical data. Proc. VLDB Endow. 8, 10 (June 2015), 986–997. https://doi.org/10.14778/2794367.2794369

[28] Michael J. Freitag, Maximilian Bandle, Tobias Schmidt, Alfons Kemper, and Thomas Neumann. 2020. Adopting Worst-Case Optimal Joins in Relational Database Systems. Proc. VLDB Endow. 13, 11 (2020), 1891–1904. http://www.vldb.org/pvldb/vol13/p1891-freitag.pdf

[29] Jonathan Goldstein and Per-Åke Larson. 2001. Optimizing queries using materialized views: a practical, scalable solution. In Proceedings of the 2001 ACM SIGMOD International Conference on Management of Data (Santa Barbara, California, USA) (SIGMOD '01). Association for Computing Machinery, New York, NY, USA, 331–342. https://doi.org/10.1145/375663.375706

[30] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2021. Cardinality estimation in DBMS: a comprehensive benchmark evaluation. Proc. VLDB Endow. 15, 4 (dec 2021), 752–765. https://doi.org/10.14778/3503585.3503586

[31] Benjamin Hilprecht and Carsten Binnig. 2022. Zero-Shot Cost Models for Out-of-the-box Learned Cost Prediction. Proc. VLDB Endow. 15, 11 (2022), 2361–2374. https://doi.org/10.14778/3551793.3551799

[32] Benjamin Hilprecht, Andreas Schmidt, Moritz Kulessa, Alejandro Molina, Kristian Kersting, and Carsten Binnig. 2020. DeepDB: Learn from Data, Not from Queries! Proc. VLDB Endow. 13, 7 (mar 2020), 992–1005. https://doi.org/10.14778/3384345.3384349

[33] Yannis E Ioannidis and Eugene Wong. 1987. Query optimization by simulated annealing. In Proceedings of the 1987 ACM SIGMOD international conference on Management of data. 9–22.

[34] Matthew Jagielski, Alina Oprea, Battista Biggio, Chang Liu, Cristina Nita-Rotaru, and Bo Li. 2021. Manipulating Machine Learning: Poisoning Attacks and Countermeasures for Regression Learning. arXiv:1804.00308 [cs.CR] https://arxiv.org/abs/1804.00308

[35] Alekh Jindal and Jyoti Leeka. 2022. Query Optimizer as a Service: An Idea Whose Time Has Come! SIGMOD Rec. 51, 3 (Nov. 2022), 49–55. https://doi.org/10.1145/3572751.3572767

[36] Konstantinos Kanellis, Badrish Chandramouli, Ted Hart, and Shivaram Venkataraman. 2025. From FASTER to F2: Evolving Concurrent Key-Value Store Designs for Large Skewed Workloads. Proc. VLDB Endow. 18, 12 (Aug. 2025), 4910–4923. https://doi.org/10.14778/3750601.3750615

[37] Kyoungmin Kim, Jisung Jung, In Seo, Wook-Shin Han, Kangwoo Choi, and Jaehyok Chong. 2022. Learned Cardinality Estimation: An In-depth Study. In Proceedings of the 2022 International Conference on Management of Data (Philadelphia, PA, USA) (SIGMOD '22). Association for Computing Machinery, New York, NY, USA, 1214–1227. https://doi.org/10.1145/3514221.3526154

[38] Kyoungmin Kim, Sangoh Lee, Injung Kim, and Wook-Shin Han. 2024. ASM: Harmonizing Autoregressive Model, Sampling, and Multi-dimensional Statistics Merging for Cardinality Estimation. Proc. ACM Manag. Data 2, 1, Article 45 (mar 2024), 27 pages. https://doi.org/10.1145/3639300

[39] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter A. Boncz, and Alfons Kemper. 2018. Learned Cardinalities: Estimating Correlated Joins with Deep Learning. ArXiv abs/1809.00677 (2018). https://api.semanticscholar.org/CorpusID:52154172

[40] Evgenios M. Kornaropoulos, Silei Ren, and Roberto Tamassia. 2022. The Price of Tailoring the Index to Your Data: Poisoning Attacks on Learned Index Structures. In Proceedings of the 2022 International Conference on Management of Data (Philadelphia, PA, USA) (SIGMOD '22). Association for Computing Machinery, New York, NY, USA, 1331–1344. https://doi.org/10.1145/3514221.3517867

[41] Meghdad Kurmanji, Eleni Triantafillou, and Peter Triantafillou. 2024. Machine Unlearning in Learned Databases: An Experimental Analysis. Proc. ACM Manag. Data 2, 1, Article 49 (mar 2024), 26 pages. https://doi.org/10.1145/3639304

[42] Meghdad Kurmanji and Peter Triantafillou. 2023. Detect, Distill and Update: Learned DB Systems Facing Out of Distribution Data. Proceedings of the ACM on Management of Data 1, 1 (2023), 1–27.

[43] Per-Ake Larson, Wolfgang Lehner, Jingren Zhou, and Peter Zabback. 2007. Cardinality estimation using sample views with quality assurance. In Proceedings of the 2007 ACM SIGMOD international conference on Management of data. 175–186.

[44] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2015. How good are query optimizers, really? Proceedings of the VLDB Endowment (Nov 2015), 204–215. https://doi.org/10.14778/2850583.2850594

[45] Beibin Li, Yao Lu, and Srikanth Kandula. 2022. Warper: Efficiently Adapting Learned Cardinality Estimators to Data and Workload Drifts. In Proceedings of the 2022 International Conference on Management of Data (Philadelphia, PA, USA) (SIGMOD '22). Association for Computing Machinery, New York, NY, USA, 1920–1933. https://doi.org/10.1145/3514221.3526179

[46] Beibin Li, Yao Lu, and Srikanth Kandula. 2022. Warper: Efficiently Adapting Learned Cardinality Estimators to Data and Workload Drifts. In Proceedings of the 2022 International Conference on Management of Data (Philadelphia, PA, USA) (SIGMOD '22). Association for Computing Machinery, New York, NY, USA, 1920–1933. https://doi.org/10.1145/3514221.3526179

[47] Pengfei Li, Wenqing Wei, Rong Zhu, Bolin Ding, Jingren Zhou, and Hua Lu. 2023. ALECE: An Attention-based Learned Cardinality Estimator for SPJ Queries on Dynamic Workloads. Proc. VLDB Endow. 17, 2 (oct 2023), 197–210. https://doi.org/10.14778/3626292.3626302

[48] Yingze Li, Hongzhi Wang, and Xianglong Liu. 2024. One Seed, Two Birds: A Unified Learned Structure for Exact and Approximate Counting. Proc. ACM Manag. Data 2, 1, Article 15 (mar 2024), 26 pages. https://doi.org/10.1145/3639270

[49] Melanie Mitchell. 1998. An introduction to genetic algorithms. MIT press.

[50] Parimarjan Negi, Ziniu Wu, Andreas Kipf, Nesime Tatbul, Ryan Marcus, Sam Madden, Tim Kraska, and Mohammad Alizadeh. 2023. Robust Query Driven Cardinality Estimation under Changing Workloads. Proc. VLDB Endow. 16, 6 (Feb. 2023), 1520–1533. https://doi.org/10.14778/3583140.3583164

[51] Theofilos Petsios, Jason Zhao, Angelos D. Keromytis, and Suman Jana. 2017. SlowFuzz: Automated Domain-Independent Detection of Algorithmic Complexity Vulnerabilities. In Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (Dallas, Texas, USA) (CCS '17). Association for Computing Machinery, New York, NY, USA, 2155–2168. https://doi.org/10.1145/3133956.3134073

[52] Viswanath Poosala, Peter J. Haas, Yannis Ioannidis, and Eugene J. Shekita. 1996. Improved histograms for selectivity estimation of range predicates. International Conference on Management of Data (1996).

[53] Shipeng Qi, Bing Tong, Jiatao Hu, Heng Lin, Yue Pang, Wei Yuan, Songlin Lyu, Zhihui Guo, Ke Huang, Xujin Ba, Qiang Yin, Youren Shen, Yan Zhou, Tao Lv, Jia Li, Lei Zou, Yongwei Wu, Gábor Szárnyas, Xiaowei Zhu, Wenguang Chen, and Chuntao Hong. 2025. The LDBC Financial Benchmark: Transaction Workload. Proc. VLDB Endow. 18, 9 (May 2025), 3007–3020. https://doi.org/10.14778/3746405.3746424

[54] D Sun, W Dong, and K Yi. 2023. Confidence Intervals for Private Query Processing. In Proceedings of the VLDB Endowment, Vol. 17. VLDB, 373–385.

[55] Jiayi Wang, Chengliang Chai, Jiabin Liu, and Guoliang Li. 2021. FACE: A normalizing flow based cardinality estimator. Proceedings of the VLDB Endowment 15, 1 (2021), 72–84.

[56] Xiaoying Wang, Changbo Qu, Weiyuan Wu, Jiannan Wang, and Qingqing Zhou. 2021. Are We Ready for Learned Cardinality Estimation? Proc. VLDB Endow. 14, 9 (may 2021), 1640–1654. https://doi.org/10.14778/3461535.3461552

[57] Johannes Wehrstein, Timo Eckmann, Roman Heinrich, and Carsten Binnig. 2025. JOB-Complex: A Challenging Benchmark for Traditional & Learned Query Optimization. https://aidb-workshop.github.io/2025/ AIDB Co-located with VLDB 2025.

[58] Peizhi Wu and Gao Cong. 2021. A Unified Deep Model of Learning from both Data and Queries for Cardinality Estimation. In Proceedings of the 2021 International Conference on Management of Data. 2009–2022.

[59] Ziniu Wu, Parimarjan Negi, Mohammad Alizadeh, Tim Kraska, and Samuel Madden. 2023. FactorJoin: A New Cardinality Estimation Framework for Join Queries. Proc. ACM Manag. Data 1, 1, Article 41 (may 2023), 27 pages. https://doi.org/10.1145/3588721

[60] Ziniu Wu, Amir Shaikhha, Rong Zhu, Kai Zeng, Yuxing Han, and Jingren Zhou. 2021. BayesCard: Revitilizing Bayesian Frameworks for Cardinality Estimation. arXiv:2012.14743 [cs.DB] https://arxiv.org/abs/2012.14743

[61] Han Xiao, Huang Xiao, and Claudia Eckert. 2012. Adversarial label flips attack on support vector machines. In Proceedings of the 20th European Conference on Artificial Intelligence (Montpellier, France) (ECAI'12). IOS Press, NLD, 870–875.

[62] Chaofei Yang, Qing Wu, Hai Li, and Yiran Chen. 2017. Generative Poisoning Attack Method Against Neural Networks. arXiv:1703.01340 [cs.CR] https://arxiv.org/abs/1703.01340

[63] Rui Yang, Evgenios M. Kornaropoulos, and Yue Cheng. 2024. Algorithmic Complexity Attacks on Dynamic Learned Indexes. Proc. VLDB Endow. 17, 4 (March 2024), 780–793. https://doi.org/10.14778/3636218.3636232

[64] Zongheng Yang, Amog Kamsetty, Sifei Luan, Eric Liang, Yan Duan, Xi Chen, and Ion Stoica. 2020. NeuroCard: One Cardinality Estimator for All Tables. Proc. VLDB Endow. 14, 1 (sep 2020), 61–73. https://doi.org/10.14778/3421424.3421432

[65] Zongheng Yang, Eric Liang, Amog Kamsetty, Chenggang Wu, Yan Duan, Xi Chen, Pieter Abbeel, Joseph M. Hellerstein, Sanjay Krishnan, and Ion Stoica. 2019. Deep Unsupervised Cardinality Estimation. Proc. VLDB Endow. 13, 3 (nov 2019), 279–292. https://doi.org/10.14778/3368289.3368294

[66] Xiang Yu, Chengliang Chai, Guoliang Li, and Jiabin Liu. 2022. Cost-Based or Learning-Based? A Hybrid Query Optimizer for Query Plan Selection. Proc. VLDB Endow. 15, 13 (Sept. 2022), 3924–3936. https://doi.org/10.14778/3565838.3565846

[67] Tianjing Zeng, Junwei Lan, Jiahong Ma, Wenqing Wei, Rong Zhu, Pengfei Li, Bolin Ding, Defu Lian, Zhewei Wei, and Jingren Zhou. 2024. PRICE: A Pretrained Model for Cross-Database Cardinality Estimation. https://doi.org/10.48550/arXiv.2406.01027

[68] Haozhe Zhang, Christoph Mayer, Mahmoud Abo Khamis, Dan Olteanu, and Dan Suciu. 2025. LpBound: Pessimistic Cardinality Estimation Using Lp-Norms of Degree Sequences. Proc. ACM Manag. Data 3, 3, Article 184 (June 2025), 27 pages. https://doi.org/10.1145/3725321

[69] Jintao Zhang, Chao Zhang, Guoliang Li, and Chengliang Chai. 2024. PACE: Poisoning Attacks on Learned Cardinality Estimation. Proc. ACM Manag. Data 2, 1, Article 37 (mar 2024), 27 pages. https://doi.org/10.1145/3639292

[70] Rui Zhang and Quanyan Zhu. 2017. A game-theoretic analysis of label flipping attacks on distributed support vector machines. In 2017 51st Annual Conference on Information Sciences and Systems (CISS). 1–6. https://doi.org/10.1109/CISS.2017.7926118

[71] Yihang Zheng, Chen Lin, Xian Lyu, Xuanhe Zhou, Guoliang Li, and Tianqing Wang. 2024. Robustness of Updatable Learning-based Index Advisors against Poisoning Attack. Proc. ACM Manag. Data 2, 1, Article 10 (March 2024), 26 pages. https://doi.org/10.1145/3639265

[72] Yingli Zhou, Yixiang Fang, Qingshuo Guo, Chenhao Ma, Yi Yang, and Laks VS Lakshmanan. 2024. In-depth Analysis of Densest Subgraph Discovery in a Unified Framework. In Proceedings of the International Conference on Very Large Data Bases (VLDB). VLDB Endowment.

[73] Yingli Zhou, Yixiang Fang, Wensheng Luo, and Yunming Ye. 2023. Influential Community Search over Large Heterogeneous Information Networks. Proc. VLDB Endow. 16, 8 (April 2023), 2047–2060. https://doi.org/10.14778/3594512.3594532

[74] Yingli Zhou, Qingshuo Guo, Yixiang Fang, and Chenhao Ma. 2024. A Counting-based Approach for Efficient k-Clique Densest Subgraph Discovery. Proc. ACM Manag. Data 2, 3, Article 119 (May 2024), 27 pages. https://doi.org/10.1145/3654922

[75] Rong Zhu, Ziniu Wu, Yuxing Han, Kai Zeng, Andreas Pfadler, Zhengping Qian, Jingren Zhou, and Bin Cui. 2021. FLAT: fast, lightweight and accurate method for cardinality estimation. Proc. VLDB Endow. 14, 9 (May 2021), 1489–1502. https://doi.org/10.14778/3461535.3461539

Due to space limitations, we present some proof details in this appendix.

APPENDIX ROADMAP

Due to space limitations in the main content, we present supplementary materials in this appendix, organized as follows:

- **Theoretical Proofs (Section A – E):** We provide the complete mathematical derivations and proofs to support our theoretical claims. This includes the NP-hardness of DSA (Theorem 3.2), the detailed derivation of the objective function (Theorem 4.1), the modularity property (Theorem 4.2), and the approximation guarantees for our algorithm (Corollary 1 & 2).
- **Experimental Settings (Section F):** To ensure reproducibility, we detail the specific hardware environments and the non-default PostgreSQL configurations used in our evaluation.
- **Extended Utility Analysis (Section G – I):** We present further investigations into system robustness, including an analysis of heuristic defenses (the "Merge Join Trap"), the potential of noise injection for hardening estimators, and detailed settings for the heavy-hitter maintenance audit.

A PROOF OF THEOREM3.2

We present the proof of Theorem 3.2 here:

Proof. We aim to prove that when $x > K \times M$, the benefit in Qerror resulting from inserting I tuples ($2 \le I \le K$) into the database constructed from Theorem 3.1 is smaller than the benefit obtained by not deleting any two tuples corresponding to any q_x. Consequently, the plan of inserting I tuples can be disregarded and replaced with I delete operations. And we can therefore reuse the PTIME reduction in theorem3.1, construct a polynomial-time reduction from the DKS problem to the DSA problem when simultaneously considering insertions and deletions. Therefore the DSA problem considering both insertions and deletions remains NP-Hard.

For a fixed query q_x, deleting its two tuples in R_x yields a benefit $g_{\text{Del}} = 2x + 1$. Given that $x > K \times M$, we have

$$g_{\text{Del}} > 2 \times K \times M + 1.$$

Assume that all M queries share one common test tuple in R_x. Therefore, repeatedly inserting into this special tuple would be the optimal solution for inserting I tuples, which provides an upper bound on the Qerror when inserting I tuples. That is,

$$g_{\text{Ins}} \le M \times \frac{I+2}{2}.$$

Given that $g_{\text{Del}} > 2 \times K \times M + 1$ and $K \ge I$, we have

$$g_{\text{Del}} > 2 \times K \times M > 2 \times I \times M = \frac{IM}{2} + \frac{3IM}{2}.$$

Since $I \ge 2$, we have $\frac{3IM}{2} > M$, and thus

$$2 \times I \times M = \frac{IM}{2} + \frac{3IM}{2} > \frac{IM}{2} + M \ge g_{\text{Ins}}.$$

In conclusion, we deduce that the benefit g_{Del} obtained by deleting any two tuples corresponding to query q_x in R_x is greater than the benefit g_{Ins} of inserting I tuples ($2 \le I \le K$). □

B PROOF OF THE DERIVATION IN THEOREM4.1

Given that $\mathbb{Q}_j(X) = \frac{C_D(\mathbf{q}_j)+1}{C_D(\mathbf{q}_j)+1+\sum_{t_i \in X} t_i.\beta \times w_{ij}}$ and $w'_{ij} = \frac{w_{ij}}{1+C_D(\mathbf{q}_j)}$, $S_j(X) = \sum_{t_i \in X} t_i.\beta w'_{ij}$

We prove that the expression

$$\mathbb{Q}_j(A \cup B) + \mathbb{Q}_j(A \cap B) - \mathbb{Q}_j(A) - \mathbb{Q}_j(B)$$

can be reorganized in the following form:

$$\frac{(2 + S_j(A) + S_j(B))(S_j(A \setminus B))(S_j(B \setminus A))}{(1 + S_j(A \cup B))(1 + S_j(A \cap B))(1 + S_j(A))(1 + S_j(B))}$$

Proof. Based on the weights w'_{ij} and the function $S_j(X)$, we define the function $\mathbb{Q}_j(X)$ as follows:

$$\mathbb{Q}_j(X) = \frac{C_D(\mathbf{q}_j) + 1}{C_D(\mathbf{q}_j) + 1 + \sum_{t_i \in X} t_i.\beta \times w_{ij}} = \frac{1}{1 + S_j(X)}.$$

Therefore, for any sets A and B, the following relationship holds:

$$\begin{aligned}&\mathbb{Q}_j(A \cup B) + \mathbb{Q}_j(A \cap B) - \mathbb{Q}_j(A) - \mathbb{Q}_j(B)\\ =&\frac{1}{1 + S_j(A \cup B)} + \frac{1}{1 + S_j(A \cap B)} - \frac{1}{1 + S_j(A)} - \frac{1}{1 + S_j(B)}\end{aligned}$$

We examine the first two terms of the above summation:

$$\frac{1}{1 + S_j(A \cup B)} + \frac{1}{1 + S_j(A \cap B)}$$

This can be combined as follows:

$$\begin{aligned}&\frac{1}{1 + S_j(A \cup B)} + \frac{1}{1 + S_j(A \cap B)}\\ =&\frac{(2 + S_j(A \cup B) + S_j(A \cap B)) \times (1 + S_j(A)) \times (1 + S_j(B))}{(1 + S_j(A \cup B))(1 + S_j(A \cap B))(1 + S_j(A))(1 + S_j(B))}\\ =&\frac{2 + S_j(A \cup B) + S_j(A \cap B)}{(1 + S_j(A \cup B))(1 + S_j(A \cap B))} \times \frac{(1 + S_j(A))(1 + S_j(B))}{(1 + S_j(A))(1 + S_j(B))}\end{aligned}$$

Similarly, the last two terms of the summation are:

$$\frac{1}{1 + S_j(A)} + \frac{1}{1 + S_j(B)}$$

These terms can be combined as:

$$\begin{aligned}&\frac{1}{1 + S_j(A)} + \frac{1}{1 + S_j(B)}\\ =&\frac{(2 + S_j(A) + S_j(B)) \times (1 + S_j(A \cup B)) \times (1 + S_j(A \cap B))}{(1 + S_j(A))(1 + S_j(B))(1 + S_j(A \cup B))(1 + S_j(A \cap B))}\\ =&\frac{2 + S_j(A) + S_j(B)}{(1 + S_j(A))(1 + S_j(B))} \times \frac{(1 + S_j(A \cup B))(1 + S_j(A \cap B))}{(1 + S_j(A \cup B))(1 + S_j(A \cap B))}\end{aligned}$$

Notably, we observe that:

$$2+S_j(A)+S_j(B) = 2+S_j(A\setminus B)+2S_j(A\cap B)+S_j(B\setminus A) = 2+S_j(A\cup B)+S_j(A\cap B)$$

Therefore, by combining the above results, we obtain:

$$\frac{1}{1+S_j(A\cup B)}+\frac{1}{1+S_j(A\cap B)}-\frac{1}{1+S_j(A)}-\frac{1}{1+S_j(B)}=\frac{(2+S_j(A)+S_j(B))\left((1+S_j(A))(1+S_j(B))-(1+S_j(A\cup B))(1+S_j(A\cap B))\right)}{(1+S_j(A\cup B))(1+S_j(A\cap B))(1+S_j(A))(1+S_j(B))}$$

Additionally, we observe that:

$$\begin{aligned}&(1+S_j(A))(1+S_j(B))-(1+S_j(A\cup B))(1+S_j(A\cap B))\\&=\left[1+S_j(A\cap B)+S_j(A\setminus B)\right]\left[1+S_j(A\cap B)+S_j(B\setminus A)\right]\\&\quad-\left[1+S_j(A\cap B)+S_j(A\setminus B)+S_j(B-A)\right](1+S_j(A\cap B))\\&=S_j(A\setminus B)\times S_j(B\setminus A).\end{aligned}$$

Therefore, we can substitute the equation in and have:

$$\begin{aligned}&\frac{1}{1+S_j(A\cup B)}+\frac{1}{1+S_j(A\cap B)}-\frac{1}{1+S_j(A)}-\frac{1}{1+S_j(B)}\\&=\frac{(2+S_j(A)+S_j(B))(S_j(A\setminus B))(S_j(B\setminus A))}{(1+S_j(A\cup B))(1+S_j(A\cap B))(1+S_j(A))(1+S_j(B))}.\end{aligned}$$

Therefore we proofed that:

$$\begin{aligned}&\mathbb{Q}_j(A\cup B)+\mathbb{Q}_j(A\cap B)-\mathbb{Q}_j(A)-\mathbb{Q}_j(B)\\&=\frac{(2+S_j(A)+S_j(B))(S_j(A\setminus B))(S_j(B\setminus A))}{(1+S_j(A\cup B))(1+S_j(A\cap B))(1+S_j(A))(1+S_j(B))}.\end{aligned}$$

□

C PROOF OF THEOREM4.2

We present the proof of Theorem 4.2 here:

Proof. We aim to demonstrate that the objective

$$\mathbb{Q}'(X)=\sum_{j=1}^{M}\mathbb{Q}_j(X)=\sum_{j=1}^{M}\frac{C_D(\mathbf{q}_j)+1+\sum_{t_i\in X}t_i.\beta\times w_{ij}}{C_D(\mathbf{q}_j)+1}$$

Due to the linear nature of the target $\mathbb{Q}'(X)$, we can move the summation over $t_i.\beta$ from the inner layer to the outer layer, rearrange $t_i.\beta$, and combine elements based on the sets $(A\setminus B)$, $(B\setminus A)$, and $(A\cap B)$. Then, we prove that $\mathbb{Q}'(X)$ satisfies the modular property.

We can rewrite $\mathbb{Q}'(X)$ as:

$$\mathbb{Q}'(X)=M+\sum_{j=1}^{M}\frac{\sum_{t_i\in X}t_i.\beta\times w_{ij}}{C_D(\mathbf{q}_j)+1}.$$

Let us define $H_i=\sum_{j=1}^{M}\frac{w_{ij}}{C_D(\mathbf{q}_j)+1}$. Consequently, we have

$$\mathbb{Q}'(X)=M+\sum_{i=1}^{N}t_i.\beta\times H_i.$$

Considering the left-hand side, we obtain:

$$\mathbb{Q}'(A\cup B)+\mathbb{Q}'(A\cap B)=\sum_{t_k\in A\cup B}t_k.\beta\times H_k+\sum_{t_k\in A\cap B}t_k.\beta\times H_k.$$

This can be expanded as:

$$\sum_{t_k\in A\setminus B}t_k.\beta\times H_k+2\sum_{t_k\in A\cap B}t_k.\beta\times H_k+\sum_{t_k\in B\setminus A}t_k.\beta\times H_k.$$

On the other hand, the right-hand side is:

$$\mathbb{Q}'(A)+\mathbb{Q}'(B)=\sum_{t_k\in A}t_k.\beta\times H_k+\sum_{t_k\in B}t_k.\beta\times H_k.$$

This also expands to:

$$\sum_{t_k\in A\setminus B}t_k.\beta\times H_k+2\sum_{t_k\in A\cap B}t_k.\beta\times H_k+\sum_{t_k\in B\setminus A}t_k.\beta\times H_k.$$

Therefore, we have

$$\mathbb{Q}'(A)+\mathbb{Q}'(B)=\mathbb{Q}'(A\cup B)+\mathbb{Q}'(A\cap B).$$

This equality demonstrates the modular property. □

D PROOF OF COROLLARY 1

Here, we will prove our Corollary 1. When the auditor specifies which queries to insert and which to delete, our optimization goal becomes

$$\begin{aligned}\text{Max}::&\sum_{\mathbf{q}_j\in\text{Insert}}\frac{C_D(\mathbf{q}_j)+1+\sum_{i=1}^{N}t_i\cdot\beta\times w_{ij}}{C_D(\mathbf{q}_j)+1}\\&+\sum_{\mathbf{q}_j\in\text{Delete}}\frac{C_D(\mathbf{q}_j)+1}{C_D(\mathbf{q}_j)+1+\sum_{i=1}^{N}t_i\cdot\beta\times w_{ij}}\\\text{s.t.}\quad&\sum_{i=1}^{N}|t_i\cdot\beta|\le K,\quad t_i\cdot\beta\in\{-1,0,1,\ldots,K\}\end{aligned}$$

According to Theorem 4.1 and Theorem 4.2, the first term of the optimization objective

$$\sum_{\mathbf{q}_j\in\text{Insert}}\frac{C_D(\mathbf{q}_j)+1+\sum_{i=1}^{N}t_i\cdot\beta\,w_{ij}}{C_D(\mathbf{q}_j)+1},\tag{6}$$

exhibits modular properties. In contrast, the second term of the optimization objective

$$\sum_{\mathbf{q}_j\in\text{Delete}}\frac{C_D(\mathbf{q}_j)+1}{C_D(\mathbf{q}_j)+1+\sum_{i=1}^{N}t_i\cdot\beta\,w_{ij}},\tag{7}$$

demonstrates supermodular characteristics. The combination exhibits supermodular characteristics. This aligns with the conditions outlined in [11], where the objective is defined as the sum of a supermodular function and a submodular function with zero curvature. Furthermore, Bai et al. [11] have demonstrated that this linear combination does not impact the approximation ratio of the greedy algorithm when applied to such linear summation functions. As a result, the approximation ratio remains:

$$\mathbb{Q}(\Delta D)\ge(1-\kappa)\mathbb{Q}(O).\tag{8}$$

E PROOF OF COROLLARY 2

We aim to demonstrate the following:

1. Under the given conditions, the optimal solution does not include insertions.

Consider the scenario where insertions are contemplated. The upper bound of the potential gain from insertions arises from the repeated insertion across M queries, specifically denoted as $K\times M$. This gain is inferior compared to the cost incurred by deleting one

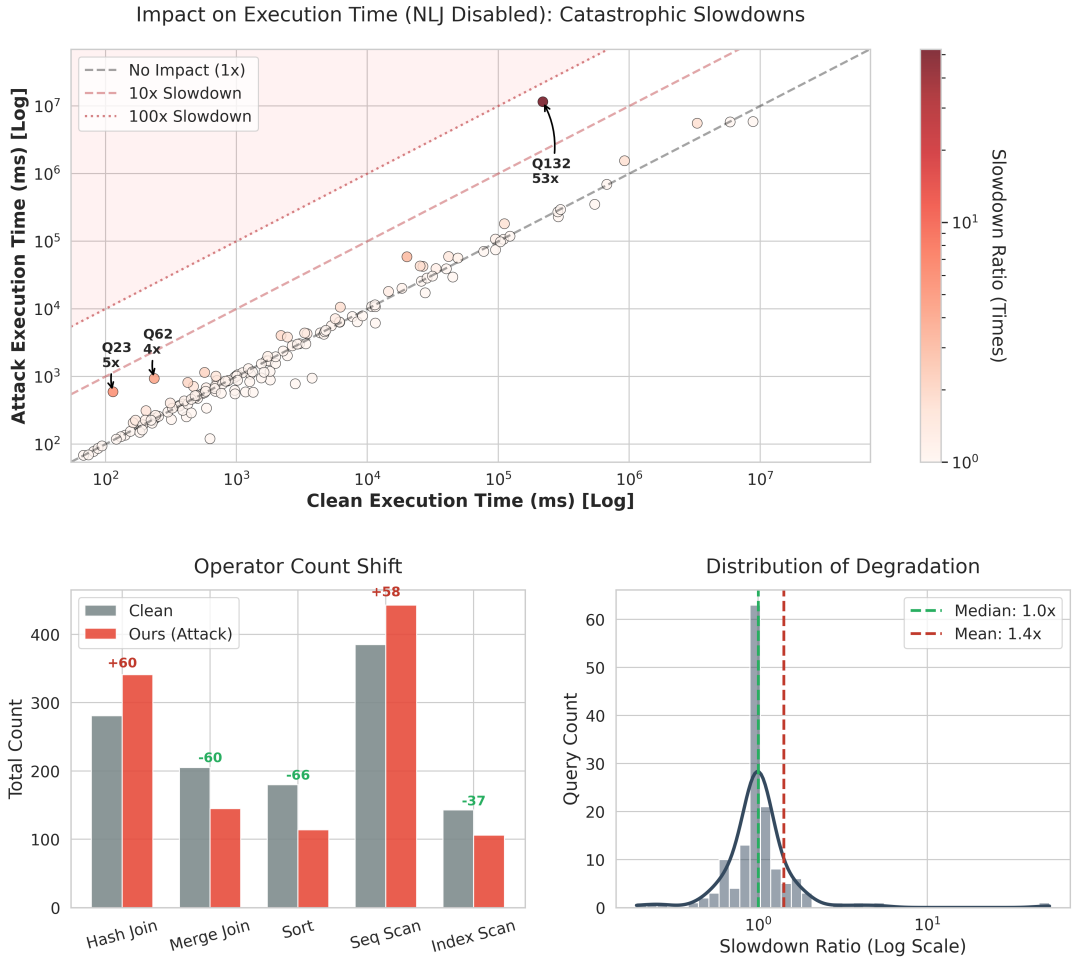


Figure 11: The "Merge Join Trap": Performance degradation audit with Nested Loop Joins disabled. (Top-Left) The DSA-perturbed estimator (Ours) forces 53× regressions even without NLJs. (Bottom-Left) The failure mechanism shifts from NLJ to expensive Merge Joins and Sorts. (Bottom-Right) Distribution of degradation.

or two tuples corresponding to a query q_x, represented as $C_D(q_j)$. Mathematically, this relationship can be expressed as:

$$K \times M < C_D(q_j)$$

Consequently, the optimal solution favors deletion over insertion, implying that the optimal solution does not incorporate insertions.

2. Under the given conditions, Algorithm 1's will not choose to insert

Algorithm 1 evaluates the remaining undeleted tuples at each step. Consider the L-th step of the algorithm, where we examine an undeleted tuple t_i. We use D_L to represent the current database state and let S_1 denote the set of queries q_j involving t_i with $count(w_{i,j} \neq 0) = 1$, S_2 denote the set of queries q_j involving t_i with $count(w_{i,j} \neq 0) = 2$. The potential gain from repeatedly inserting tuples associated with queries in S is given by:

$$gainI = \sum_{q_j \in S_1} \frac{2 \cdot C_{D_L}(q_j) + 1}{C_{D_L}(q_j) + 1} + \sum_{q_j \in S_2} \frac{C_{D_L}(q_j) + 1 + w_{i,j}}{C_{D_L}(q_j) + 1}$$

On the other hand, if tuple t_i is deleted, the corresponding gain is:

$$gainD = \sum_{q_j \in S_1} \frac{C_{D_L}(q_j) + 1}{1} + \sum_{q_j \in S_2} \frac{C_{D_L}(q_j) + 1}{C_{D_L}(q_j) + 1 - w_{i,j}}$$

Note that:
$\frac{C_{D_L}(q_j)+1}{C_{D_L}(q_j)+1-w_{i,j}} - \frac{C_{D_L}(q_j)+1+w_{i,j}}{C_{D_L}(q_j)+1} = \frac{w_{i,j}}{C_{D_L}(q_j)+1-w_{i,j}} - \frac{w_{i,j}}{C_{D_L}(q_j)+1} > 0$
and:

$$\frac{C_{D_L}(q_j) + 1}{1} - \frac{2 \cdot C_{D_L}(q_j) + 1}{C_{D_L}(q_j) + 1} = C_{D_L}(q_j) - \frac{C_{D_L}(q_j)}{C_{D_L}(q_j) + 1} \geq 0$$

We can compute the difference between the inner summations of the two groups separately, yielding:

$$\sum_{q_j \in S_1} \frac{C_{D_L}(q_j)+1}{1} - \sum_{q_j \in S_1} \frac{2 \cdot C_{D_L}(q_j)+1}{C_{D_L}(q_j)+1}$$
$$= \sum_{q_j \in S_1} \left(\frac{C_{D_L}(q_j)+1}{1} - \frac{2 \cdot C_{D_L}(q_j)+1}{C_{D_L}(q_j)+1} \right) > 0$$

Additionally, for the second group, we have:

$$\sum_{q_j \in S_2} \frac{C_{D_L}(q_j)+1}{C_{D_L}(q_j)+1-w_{i,j}} - \sum_{q_j \in S_2} \frac{C_{D_L}(q_j)+1+w_{i,j}}{C_{D_L}(q_j)+1}$$
$$= \sum_{q_j \in S_2} \left(\frac{C_{D_L}(q_j)+1}{C_{D_L}(q_j)+1-w_{i,j}} - - \frac{C_{D_L}(q_j)+1+w_{i,j}}{C_{D_L}(q_j)+1} \right) > 0$$

Combining the results from both groups, we obtain:

$$gainI < gainD$$

This inequality indicates that the gain from insertion (*gainI*) is less than the gain from deletion (*gainD*). Therefore, Algorithm 1 will opt for deletion over insertion at each step. As a result, the algorithm inherently avoids insertions, effectively reducing the problem to an insert-only scenario under the given conditions.

$$\mathbb{Q}(\Delta D) \geq (1-\kappa)\mathbb{Q}(O). \tag{9}$$

F DETAILED EXPERIMENTAL SETTINGS

To ensure the reproducibility of our results and to provide a fair comparison against state-of-the-art baselines, we provide a comprehensive disclosure of our hardware infrastructure and the specific database configurations used in our end-to-end evaluation.

F.1 Hardware Environments

We decoupled the model training phase from the query execution phase to optimize resource utilization appropriate for each task. The hardware specifications are detailed as follows:

- **End-to-End Query Evaluation Platform:** All end-to-end latency experiments, including the integration of learned estimators into the PostgreSQL kernel, were conducted on an individual Huawei Cloud instance. This environment features **4 Intel(R) Xeon(R) Platinum 8378A CPUs**, **32GB of RAM**, and a **1TB SSD**. This configuration was chosen to simulate a realistic, resource-constrained database server environment where I/O and CPU efficiency are critical.
- **Model Training Platform:** The training of deep learning-based cardinality estimators (and other baselines requiring GPU acceleration) was performed on a high-performance computing server equipped with **4 NVIDIA RTX A6000 GPUs**, **40 Intel(R) Xeon(R) Silver 4210R CPUs**, and **504GB of RAM**.

F.2 Software and Database Configuration

For the database system, we utilized **PostgreSQL 13.1**. To maintain strict objectivity and facilitate direct comparisons with prior work, our experimental environment is derived from the standard docker image provided by the *Cardinality Estimation Benchmark* (CEB). Using this standardized container ensures that our operating system dependencies and libraries are consistent with established benchmarks in the community.

F.2.1 PostgreSQL Parameter Settings. We report the specific PostgreSQL configurations used during the runtime evaluation in Table 3. Note that we list only the parameters that differ from the default PostgreSQL configuration (mode: `changed`).

Crucially, to isolate the impact of cardinality estimation quality on query performance, we disabled parallel query execution (setting `max_parallel_workers` and related parameters to 0) and the Genetic Query Optimizer (`geqo = off`). This ensures that the reported end-to-end latencies reflect the efficiency of the physical plans generated by the optimizer, rather than variances introduced by thread scheduling or nondeterministic planning heuristics.

Table 3: PostgreSQL 13.1 Configuration Parameters (Non-default)

Parameter Name	Value	Unit	Description
Resource Usage & Memory			
shared_buffers	524288	8kB	Shared memory buffers (approx. 4GB)
work_mem	4096	kB	(Implicit default in CEB environment)
wal_buffers	2048	8kB	Disk-page buffers in shared memory for WAL
max_stack_depth	2048	kB	Maximum stack depth
dynamic_shared_memory_type	posix	-	Dynamic shared memory implementation
Planner & Optimizer (Crucial for Reproducibility)			
geqo	off	-	**Disable Genetic Query Optimization**
max_parallel_workers	0	-	**Disable Parallel Execution**
max_parallel_workers_per_gather	0	-	Disable parallel processes per node
default_text_search_config	pg_catalog.english	-	Default text search configuration
Write Ahead Log (WAL) & Checkpoints			
max_wal_size	51200	MB	WAL size triggering a checkpoint
min_wal_size	80	MB	Minimum WAL size
wal_segment_size	16777216	B	Size of WAL segments
archive_command	(disabled)	-	Command to archive WAL files
Connections & Networking			
max_connections	100	-	Max concurrent connections
listen_addresses	*	-	Listen on all IP addresses
port	5432	-	Listening port
tcp_keepalives_count	9	-	Max TCP keepalive retransmits
tcp_keepalives_idle	7200	s	Time between TCP keepalives
tcp_keepalives_interval	75	s	Interval between keepalive retransmits
unix_socket_permissions	0777	-	Access permissions for Unix socket
Locale, Formatting & Paths			
DateStyle	ISO, MDY	-	Date and time display format
TimeZone	Etc/UTC	-	Time zone for timestamps
lc_collate	C	-	Collation order locale
lc_ctype	C	-	Character classification locale
server_encoding	SQL_ASCII	-	Database character set encoding
data_directory	/var/lib/pgsql/13.1/data	-	Server data directory
config_file	.../postgresql.conf	-	Main configuration file path

G UTILITY ANALYSIS: THE POTENTIAL& LIMITS OF OPERATOR RESTRICTION

Our primary evaluation (§ 5.2) revealed that many of the catastrophic performance regressions identified by DSA were associated with the optimizer erroneously selecting *Nested Loop Joins* (NLJ) for large intermediate results. This observation suggests a potential heuristic defense: *Can we immunize the system against worst-case data drifts simply by disabling risky operators like NLJ?*

To evaluate the efficacy of this "Safety-First" configuration, we re-conducted the DSA audit on the STATS-CEB benchmark with NLJs explicitly disabled (`enable_nestloop = off`).

G.1 Average-Case Mitigation vs. Tail Latency Failure

The results, visualized in Figure 11, present a nuanced finding. On one hand, enforcing this operator restriction successfully mitigates the impact of estimation errors for the median query, preventing the naive $O(M \times N)$ complexity explosions seen in the baseline settings.

However, the DSA reveals that this heuristic defense is insufficient for the **tail distribution**. As shown in Figure 11 (Top-Left), specific data drifts still trigger catastrophic slowdowns, with the worst-case query (Q132) degrading by **52.77**×. This demonstrates that while disabling NLJs acts as a "guardrail" for average cases, it cannot compensate for the fundamental blindness of the estimator regarding specific data correlations.

G.2 Mechanism of Defense Failure: The "Merge Join Trap"

By analyzing the execution plans generated under this restrictive policy, we identified a phenomenon we term the **"Merge Join Trap."** When the estimator drastically underestimates cardinality (due to the DSA-targeted drift), and the "easy" path of NLJ is blocked, the optimizer seeks the *next cheapest* alternative based on its flawed cost model.

Typically, it erroneously predicts that the intermediate results are small enough to be sorted and merged efficiently. In reality, the massive data volume triggers disk-spilling `Sort` operations and high-overhead `Merge Joins`. This confirms that the root cause of fragility is not the choice of physical operator, but the *data-driven belief system* of the estimator itself.

G.3 Case Study: STATS-Q132

Listing 1 illustrates this failure mode. Even with the "Safety-First" configuration (Plan B), the perturbed estimator misleads the optimizer into a path that is theoretically safe but practically disastrous.

```
-- [A] OPTIMAL PLAN (Clean Data, No NLJ)
-- Runtime: 219s | Strategy: Hash Join
-> Aggregate
   ...
   -> Hash Join (p.OwnerUserId = u.Id)
      -- Optimizer correctly sees high selectivity
      -> Seq Scan on posts p (Filter: AnsCount<=5)
      -> Hash (users u)

-- [B] FAILED DEFENSE PLAN (DSA Perturbed, No NLJ)
-- Runtime: 11,562s | Cost: ~7,093
-- Pathology: The "Merge Join Trap"
-> Aggregate
   -> Merge Join (v.UserId = u.Id)
      -> Merge Join (v.UserId = ph.UserId)
         -- < Est. Rows: Low -> Choice: Merge >
         -> Merge Join (p.OwnerUserId = v.UserId)
            -> Sort (Key: p.OwnerUserId)
            -> Materialize
         -- < REALITY: Intermediate result explodes >
         -- < Actual: 3.8 BILLION rows >
         -> Index Scan on postHistory ph
```

Listing 1: Comparison of Plans under "Safety-First" Configuration (No NLJ). Even without NLJs, the estimator forces a fallback to a pathological Merge Join.

Takeaway: *Strategic Insight: Beyond Heuristic Defenses:*This experiment highlights the unique value of **Data-centric Sensitivity Analysis (DSA)**. It proves that reactive heuristics—such as disabling risky operators—are essentially "band-aids" that mask symptoms but do not cure the disease. DSA enables us to move beyond these superficial fixes by exposing the **theoretical robustness limits** of the estimator. It suggests that next-generation robust optimizers must not only restrict operators but also incorporate **uncertainty-aware planning** that can detect and mitigate the specific data-level blind spots revealed by our audit.

H UTILITY ANALYSIS: FURTHER EXPLORATIONS

Following the utility analysis in Section 5.5, this section provides additional explorations into how DSA identifies system vulnerabilities and evaluates potential mechanisms for enhancing the robustness of learned cardinality estimators.

H.1 Hardening Estimators via Controlled Noise Injection

As demonstrated by our DSA auditor, learned models often exhibit significant performance regressions when encountering specific "heavy-hitter" data distributions. To mitigate these worst-case failures, we investigate a robustness-enhancing strategy inspired by uncertainty-aware mechanisms in differential privacy [24, 25, 54]. By systematically introducing controlled perturbations to estimator outputs, we can mask the fine-grained data characteristics that trigger model instability, effectively creating a "buffer zone" against distribution shifts.

Formally, we modify the estimator output to generate a more robust prediction:

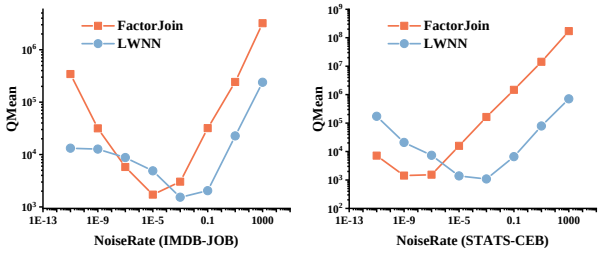
$$\text{Est}_{\text{Robust}} = \text{Est} + |\alpha \cdot \eta| \tag{10}$$

where σ denotes the standard deviation of estimator outputs, α is a scaling factor for robustness intensity, and $\eta \sim \mathcal{N}(0, \sigma^2)$ represents Gaussian noise. This approach establishes an $\alpha\sigma$-radius protection boundary, shielding the downstream optimizer from highly sensitive (and potentially erroneous) cardinality estimates.

Figure 12 illustrates the impact of this hardening strategy across different estimators and datasets. Our diagnostic analysis reveals three key insights into system robustness:

- **Robustness Gains:** Strategic noise injection (reducing Q-Mean by up to three orders of magnitude) successfully mitigates the catastrophic failures identified by our DSA auditor, stabilizing the estimator's performance under stress.
- **Configuration Sensitivity:** The optimal robustness parameter α varies significantly across datasets and model architectures. This highlights that "blind" hardening is insufficient; precise auditing via DSA is required to determine the appropriate intensity.
- **The Stability-Precision Tradeoff:** Excessive perturbation ($\alpha > 1.0$) leads to model "collapse," where the error introduced by the hardening mechanism itself exceeds the risks posed by data drift. This underscores the importance

of the DSA framework in finding the "sweet spot" between nominal precision and worst-case stability.



(a) IMDB-JOB Dataset (b) STATS-CEB Dataset

Figure 12: Evaluating Robustness Enhancement via Controlled Noise Injection.

Strategic Insights: Our investigation confirms that building robust database systems requires a rigorous understanding of an estimator's vulnerability profile. The DSA framework provides the theoretical and empirical foundation to: (1) Characterize the "robustness surface" of learned components; (2) Calibrate noise-response curves for automated system tuning; and (3) Establish practical bounds for the tradeoff between estimation accuracy and execution-plan stability.

I DETAILED SETTINGS FOR ROBUST MODEL MAINTENANCE AUDITING

In this section, we provide the technical specifics and logical motivation for the proactive maintenance experiment discussed in Section 5.5.

I.1 Heavy-Hitter Update Scenario Modeling

To rigorously test the "Early Warning" capability of DSA, we simulate a *worst-case yet realistic* dynamic environment. Traditional evaluations often assume uniform or random data updates; however, real-world database activity is frequently skewed toward specific "Heavy Hitter" tuples—data points that are both frequently updated and highly relevant to join-intensive queries (e.g., trending posts in the STATS-CEB dataset).

We perform 20 consecutive rounds of updates on the `posts` table. In each round, we modify 1% of the total tuples. Crucially, these modifications are not random but target the top-1% of tuples with the highest joint weights, creating a "correlated drift" that is particularly lethal to outdated learned estimators. This setup represents a scenario where a small but significant shift in key data distributions can lead to catastrophic optimizer failures if not detected early.

I.2 Maintenance Strategy Specifications

We compare the *DSA-Guided Sentinel* against two traditional paradigms:

- **Reactive (Post-hoc):** This strategy mimics a system that lacks proactive monitoring. It monitors the *observed* estimation error during runtime. An update is triggered only after the average Q-error of the executed workload exceeds a threshold of 50. As shown in our results, this strategy inevitably suffers from a "detection lag," as the model is already compromised by the time the error is recorded.
- **Periodic (High Budget):** This is a conservative strategy that ignores data characteristics and retrains the model every 2% of total data updates ($U = 10$ total retrains). While stable, it incurs prohibitive overhead by performing redundant training even when the distribution shift is benign.
- **DSA-Guided Sentinel (Ours):** Our approach focuses on the *potential* impact of updates. We track the modifications of tuples within the top 25% of joint weights. Each modification event triggers the DSA auditor, which efficiently approximates the 90th-percentile (P90) Q-error across the testing workload using Algorithm 1. A retraining process is initiated only if the predicted P90 Q-error exceeds 3×10^4.

I.3 Efficiency Analysis

By focusing on the structural sensitivity of the data distribution rather than raw update volume or historical error, the DSA-Guided Sentinel identifies the *critical mass* of drift. As illustrated in Figure 7, DSA maintains a near-optimal error profile with only 4 retraining operations ($U = 4$), effectively suppressing the catastrophic error spikes seen in the Reactive approach while achieving a 60% reduction in computational budget compared to the Periodic method.

J EXTENDED SENSITIVITY ANALYSIS ON IMDB

To further validate the robustness of our budget calibration across diverse data distributions, we extend our evaluation to the `IMDB` dataset (using JOB-Light queries). IMDB is widely recognized for its challenging join correlations and non-uniform distributions, providing a rigorous testbed for auditing methods. These results, summarized in Table 4, corroborate the observations from Section 5.2 with even more pronounced performance stratification.

First, at extremely low budgets (e.g., 31,603 rows, representing only **0.05%** of the total database), traditional drift patterns fail to compromise the estimator. The *Sequential* method remains almost entirely inert (Median Q-Error ≈ 1.0), while the *Heuristic* method only induces a marginal median error of 1.90. In contrast, DSA identifies a worst-case perturbation that drives the 95th percentile error to 3.22×10^5, demonstrating that structural fragility can be exploited even when a negligible fraction of the database is modified.

Second, the data justifies the 20% local budget (0.82% of the total DB) as a reasonable "activation threshold" for baseline methods. At this level, the *Heuristic* approach begins to show non-trivial degradation (Median Q-Error of 18.6 and Max of 3.24×10^7). However, under the same budget, DSA renders the estimator essentially dysfunctional, with a median error of 1.50×10^5 and a maximum error reaching 9.54×10^9. This comparison reinforces that while baselines can eventually induce error given enough budget, they lack the precision of DSA in finding the estimator's most vulnerable regions.

Takeaways: On the IMDB dataset, the gap between targeted and heuristic drifts is even more pronounced than on STATS.

Table 4: Sensitivity Analysis and Budget Calibration on IMDB.

Drift Constraint / Budget			Audit Method	Sensitivity Metric (Q-Error)			
Rows	Table %	Total DB %		50%	90%	95%	Max
31,603	1.25%	0.05%	Sequential	1.00	1.01	1.01	1.01
			Heuristic	1.90	14.4	48.1	715
			DSA (Ours)	**9.14**	1.55×10^5	3.22×10^5	3.24×10^7
63,206	2.50%	0.10%	Sequential	1.06	1.07	1.08	1.09
			Heuristic	2.39	42.6	244	1.14×10^6
			DSA (Ours)	**16.4**	2.72×10^5	8.83×10^5	3.24×10^7
126,412	5.00%	0.20%	Sequential	1.14	1.17	1.18	1.25
			Heuristic	3.70	110	509	1.50×10^6
			DSA (Ours)	**33.2**	9.03×10^6	4.28×10^7	1.35×10^9
252,824	10.0%	0.41%	Sequential	1.44	1.59	1.83	3.20
			Heuristic	6.81	458	1.82×10^3	1.50×10^6
			DSA (Ours)	**77.9**	2.73×10^7	8.30×10^7	1.35×10^9
505,648	20.0%	0.82%	Sequential	2.15	2.44	2.77	780
			Heuristic	18.6	1.82×10^3	1.24×10^4	3.24×10^7
			DSA (Ours)	1.50×10^5	1.23×10^8	5.04×10^8	9.54×10^9

DSA reveals that cardinality estimators can be driven to catastrophic failure (95th percentile error $> 10^8$) at budgets as low as 0.82% of the total database, confirming that structural vulnerabilities are a general property of learned estimators across complex schemas.